Examensarbete inom teknik

Grundnivå, 15 hp

Hierarchical k-GNN Explanations via Generative Interpretation Networks

Comparing 1-GNN and 1-2-GNN for Model-Level Graph Classification Interpretability

Tage Sköldefors

Maximillian Nordler

Abstract

Graph Neural Networks (GNNs) are widely used for graph classification but hard to interpret. Higher-order k-GNNs, which pass messages over groups of nodes rather than single nodes, are provably more expressive than standard GNNs, yet it is unclear whether this extra expressiveness actually changes what the model learns about each class.

We compare a standard 1-GNN against a hierarchical 1-2-GNN by generating class-representative explanation graphs for each one using GIN-Graph, a Wasserstein-GAN-based framework guided by a pretrained classifier. We extend GIN-Graph with a fully differentiable dense wrapper that keeps gradients flowing through both node features and the adjacency matrix of a hierarchical k-GNN.

On MUTAG, both classifiers reach the same test accuracy (89.5%), yet their generated populations diverge sharply by class: the 1-2-GNN's Non-Mutagen graphs are halogen-heavy, while its Mutagen graphs suppress halogens and triangles. Cross-classifying 1000 generated graphs per cell confirms the divergence: each model strongly rejects the other's graphs in one specific class per dataset (12.3% and 14.0% agreement). The triangle suppression is the only theory-aligned signal we observe: pair-level message passing can distinguish triangles that standard 1-WL cannot. Same accuracy, different prototypes: the two architectures arrive at different ideas of what counts as a typical class member.

Contents

1 Introduction

Graph Neural Networks (GNNs) have become the standard approach for learning on graph-structured data, achieving strong results on tasks ranging from molecular property prediction to social network analysis. Despite their effectiveness, GNNs suffer from the same interpretability challenges as other deep learning models: their predictions are difficult to explain in human-understandable terms.

The expressiveness of standard message-passing GNNs is bounded by the 1-dimensional Weisfeiler–Leman (1-WL) graph isomorphism test. Morris et al. [Morris et al.(2019)] proposed k-GNNs that operate on k-element subsets of nodes, achieving expressiveness equivalent to the k-WL test. In theory, higher-order k-GNNs can distinguish graph structures that standard GNNs cannot.

A natural question arises: *does this additional structural expressiveness lead the model to learn different internal representations of graph classes, and can we observe these differences through generated explanations?* If a model captures richer structural patterns, the explanations it produces (representative graphs that characterise each class) may reveal what structural features each architecture has learned to associate with each class.

We investigate this question using the GIN-Graph framework [Sun et al.(2025)], which generates model-level explanation graphs through a GAN-based approach guided by a pretrained classifier. Model-level explanations aim to answer the question "what does a typical graph of class c look like according to the model?" by generating new graphs that maximise the model's confidence for that class while remaining structurally realistic. This contrasts with instance-level methods that explain individual predictions by identifying important subgraphs.

Specifically, we compare:

- A **1-GNN**: standard node-level message passing, equivalent to 1-WL;
- A **1-2-GNN**: hierarchical architecture combining node-level (1-GNN) and pairwise (2-GNN) message passing, achieving expressiveness beyond 1-WL.

This work:

1. Reuses the dense 1-GNN wrapper from GIN-Graph [Sun et al.(2025)] and applies the same logic to the 2-GNN branch of a 1-2-GNN classifier, so that GIN-Graph training can be run against a hierarchical k-GNN.
2. Computes the embedding similarity component s of the validation score via cosine similarity between generated and real graph embeddings. The GIN-Graph reference implementation uses the prediction probability p as a proxy for s; we use the actual embedding.
3. Compares the explanations produced by the two architectures on MUTAG and PROTEINS. The two models produce different explanation structures and show asymmetric cross-model classification agreement, with some configurations showing near-zero agreement.

2 Background

2.1 Graph Neural Networks

A graph $G = (V, E)$ consists of a node set V with $|V| = n$ and an edge set $E \subseteq V \times V$. Each node $v \in V$ carries a feature vector $\mathbf{x}_v \in \mathbb{R}^d$. We denote the adjacency matrix as $\mathbf{A} \in \{0,1\}^{n \times n}$ where $A_{uv} = 1$ if $(u, v) \in E$, and the feature matrix as $\mathbf{X} \in \mathbb{R}^{n \times d}$ where row v is $\mathbf{x}_v$.

GNNs learn node representations through iterative *message passing*. At each layer t, a node updates its representation by aggregating information from its neighbours:

$$\mathbf{h}_v^{(t)} = \mathrm{UPDATE}\big(\mathbf{h}_v^{(t-1)},\ \mathrm{AGGREGATE}\big(\big\{\!\big\{\mathbf{h}_u^{(t-1)} : u \in \mathcal{N}(v)\big\}\!\big\}\big)\big)\,, \tag{1}$$

where $\mathcal{N}(v) = \{u \in V : (u, v) \in E\}$ denotes the neighbourhood of v, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the initial node feature, and $\{\!\{\cdot\}\!\}$ denotes a multiset.

The choice of UPDATE and AGGREGATE functions determines the specific GNN variant. In our implementation, these are parameterised by separate linear transformations for the self-feature and the aggregated neighbour features. After T layers of message passing, a *readout* function pools all node embeddings into a single graph-level representation:

$$\mathbf{h}_G = \mathrm{READOUT}\big(\big\{\mathbf{h}_v^{(T)} : v \in V\big\}\big)\,. \tag{2}$$

Common choices for READOUT include sum, mean, and max pooling over the node set. We use sum pooling for the classifier readouts, as it preserves information about both the features and the number of nodes.

2.2 The k-WL Hierarchy and k-GNNs

Two graphs are *isomorphic* if one can be obtained from the other by relabelling nodes. The 1-dimensional *Weisfeiler–Leman* (1-WL) test decides isomorphism up to a known failure class by iteratively hashing node labels. Each node's new label is a hash of its current label together with the multiset of its neighbours' labels. After convergence, two graphs are declared possibly isomorphic iff their label histograms agree.

The standard failure mode is that 1-WL cannot tell apart two graphs in which every node has the same local signature. The canonical example is two disjoint triangles versus a single hexagon: every node is degree 2 with two degree-2 neighbours, so 1-WL assigns the same labels in both graphs at every iteration even though the graphs are not isomorphic.

The k-WL test fixes this by hashing over k-tuples of nodes rather than individual nodes, which makes non-local structural differences visible; 2-WL already distinguishes the triangles-vs-hexagon example. Morris et al. [Morris et al.(2019)] showed that 1-GNNs are at most as powerful as 1-WL in distinguishing power, and that k-GNNs operating on k-element node subsets ("k-sets") are at most as powerful as k-WL (their Propositions 1–4). This is the motivation for considering a 1-2-GNN: it augments node-level message passing with pair-level message passing and can therefore capture structural distinctions invisible to a standard 1-GNN. We rely on these results as background and do not reproduce them here.

2.3 GIN-Graph

GIN-Graph [Sun et al.(2025)] is a model-level explanation method that generates class-representative graphs using a generative adversarial approach. The key idea is to train a generator $\mathcal{G}$ to produce graphs that satisfy two objectives simultaneously:

1. **Realism**: generated graphs should be structurally similar to real graphs from the dataset, enforced by a WGAN-GP discriminator.
2. **Class specificity**: generated graphs should be confidently classified as the target class by the pretrained GNN, enforced by a classification guidance loss.

The generator maps noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to a graph $(\tilde{A}, \tilde{X})$ using Gumbel-Softmax for differentiable discrete sampling. The training objective balances the two losses through a dynamic weighting schedule that shifts focus from realism (early training) to class specificity (late training).

3 Method

This section details the mathematical formulations of our two classifier architectures (1-GNN and 1-2-GNN), the GIN-Graph generator, the training procedure, and the differentiable dense wrapper that bridges them.

3.1 1-GNN Architecture

Our 1-GNN implements message passing with separate transformations for self-features and neighbour-aggregated features. At layer t, the update rule for node v is:

$$\mathbf{h}_v^{(t)} = \sigma\left(\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}\right), \tag{3}$$

where $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)} \in \mathbb{R}^{d_{t-1} \times d_t}$ are learnable weight matrices, σ is the ReLU activation function $\sigma(x) = \max(0, x)$, and $d_0 = d$ (the input feature dimension). The self-transformation $\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)}$ allows the model to retain and transform the node's own representation, while the neighbour term $\sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}$ aggregates structural context.

This can equivalently be written in matrix form for the entire graph:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)}\mathbf{W}_1^{(t)} + \mathbf{A}\,\mathbf{H}^{(t-1)}\mathbf{W}_2^{(t)}\Big), \tag{4}$$

where $\mathbf{H}^{(t)} \in \mathbb{R}^{n \times d_t}$ is the matrix of all node representations at layer t, and $\mathbf{A} \in \{0, 1\}^{n \times n}$ is the adjacency matrix. The matrix product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ computes the sum of neighbour features for every node simultaneously.

After $T = 3$ layers with hidden dimension $d_1 = d_2 = d_3 = 64$, the graph embedding is obtained via sum pooling:

$$\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T)} \in \mathbb{R}^{d_T}. \tag{5}$$

This embedding is fed through a two-layer classifier MLP with ReLU activation and dropout:

$$\hat{y} = \mathrm{softmax}\Big(\mathbf{W}_o\,\sigma(\mathbf{W}_c\,\mathbf{h}_G^{(1)})\Big)\,, \tag{6}$$

where $\mathbf{W}_c \in \mathbb{R}^{d_T \times d_T}$ and $\mathbf{W}_o \in \mathbb{R}^{d_T \times C}$ with C being the number of classes.

3.2 2-GNN and the Hierarchical 1-2-GNN

A 1-GNN aggregates only over individual node neighbourhoods, and two graphs in which every node has the same local signature are therefore indistinguishable to it. The canonical instance is two disjoint triangles compared with a single hexagon: every node has degree two with two degree-two neighbours, so the aggregated node features coincide in both graphs. Representing the graph at the level of node pairs restores the missing structural signal, because a pair carries information about whether its two elements are adjacent and about the neighbourhoods of both elements simultaneously.

A pair feature is a fixed-dimensional vector attached to each unordered node pair that summarises the pair together with an indicator of its edge status in the original graph. For each pair $s = \{u, v\}$ with canonical ordering $u < v$, the initial pair feature is built from the two 1-GNN node embeddings together with the edge indicator:

$$\mathbf{f}_s^{(0)} = \big[\mathbf{h}_u^{(T)} \parallel \mathbf{h}_v^{(T)} \parallel A_{uv}\big] \in \mathbb{R}^{2d_T+1}, \tag{7}$$

The first two slots concatenate the 1-GNN embeddings of the two endpoints, so the pair feature carries the node-level context of both u and v. The third slot, $A_{uv} \in \{0, 1\}$, is the entry of the adjacency matrix for the pair: it is 1 if (u, v) is an edge in the original graph and 0 otherwise. This bond bit acts as the isomorphism-type marker for the pair, distinguishing bonded pairs from unbonded pairs that would otherwise share the same node-level content.

Message passing on pairs uses a neighbourhood in which each neighbour of $s = \{u, v\}$ is obtained by replacing one of its two elements with a node adjacent in G to the removed element:

$$\mathcal{N}_L(\{u, v\}) = \{\{v, w\} : w \neq u,\ (u, w) \in E\} \cup \{\{u, w\} : w \neq v,\ (v, w) \in E\}\,. \tag{8}$$

The first set replaces u by a node w adjacent to u in G; the second replaces v by a node w adjacent to v in G. The neighbourhood relation among pairs therefore inherits its structure from the edge structure of the underlying graph.

The pair-level update combines the pair's own features with an aggregation over its pair-neighbourhood through two weight matrices:

$$\mathbf{f}_s^{(\ell+1)} = \sigma\left(\mathbf{f}_s^{(\ell)}\mathbf{W}_3^{(\ell)} + \sum_{s' \in \mathcal{N}_L(s)} \mathbf{f}_{s'}^{(\ell)}\mathbf{W}_4^{(\ell)}\right), \tag{9}$$

The self-transformation $\mathbf{W}_3^{(\ell)}$ preserves the pair's own representation across layers, and the neighbour-transformation $\mathbf{W}_4^{(\ell)}$ propagates information from adjacent pairs. The 2-GNN uses $T_2 = 2$ such layers.

The hierarchical 1-2-GNN runs the 1-GNN first for $T_1 = 3$ layers to obtain node embeddings, uses these embeddings to form the initial pair features via Equation (7), and then

runs T_2 layers of pair-level message passing. The graph representation is the concatenation of a node-level and a pair-level readout:

$$\mathbf{h}_G = \left[\mathbf{h}_G^{(1)} \parallel \mathbf{h}_G^{(2)}\right] \in \mathbb{R}^{2d_T}, \tag{10}$$

The node-level readout sums the final node embeddings, $\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T_1)}$, and the pair-level readout sums the final pair features:

$$\mathbf{h}_G^{(2)} = \sum_{\{u,v\} \in \binom{V}{2}} \mathbf{f}_{\{u,v\}}^{(T_2)} \in \mathbb{R}^{d_T}. \tag{11}$$

The concatenated embedding $\mathbf{h}_G$ is passed through the classifier of Equation (6) with input dimension $2d_T$ in place of d_T.

Scalability. The number of pairs grows as $\binom{n}{2} = n(n-1)/2$, which becomes prohibitive for large graphs. For PROTEINS (up to 620 nodes, yielding up to 191,890 pairs), all pairs are processed using a numba-accelerated sparse kernel with a CSR adjacency representation, giving $O(\text{degree})$ neighbour lookup instead of the $O(N)$ cost of dense masks. The full pairwise structure is therefore computed exactly, without sampling.

3.3 GIN-Graph Generator

MLP backbone. The generator $\mathcal{G}$ maps a latent noise vector to a graph through a shared two-layer MLP backbone with two output heads, one producing adjacency logits and one producing node-feature logits:

$$\mathbf{h} = \text{LeakyReLU}(\mathbf{W}_2 \,\text{LeakyReLU}(\mathbf{W}_1 \mathbf{z})) \in \mathbb{R}^{2d_h}, \tag{12}$$

$$\tilde{A}_{\text{raw}} = \text{sym}(\text{reshape}(\mathbf{h}\,\mathbf{W}_A,\ N \times N)), \tag{13}$$

$$\tilde{X}_{\text{raw}} = \text{reshape}(\mathbf{h}\,\mathbf{W}_X,\ N \times D). \tag{14}$$

The latent vector $\mathbf{z} \in \mathbb{R}^{d_z}$ is drawn from a standard normal, with $d_z = 32$ and hidden width $d_h = 128$. The adjacency head produces an $N \times N$ matrix, where N is the maximum number of nodes in the generated graph, and the feature head produces an $N \times D$ matrix, where D is the number of node feature types in the target dataset. The activation is $\text{LeakyReLU}_\alpha(x) = \max(\alpha x, x)$ with negative slope $\alpha = 0.2$; the nonzero slope on the negative side keeps gradients flowing through units with negative pre-activations, which avoids collapsing diversity of the generator output when a subset of hidden units would otherwise become dead under ReLU. The symmetrisation $\text{sym}(\mathbf{M}) = \frac{1}{2}(\mathbf{M} + \mathbf{M}^\top)$ enforces undirected adjacency. The outputs of Equations (13)–(14) are real-valued logits, not yet a graph; a discrete graph is obtained in the following step.

Gumbel-Softmax sampling. The quantities the generator must produce are discrete: each edge is binary, and each node type is a choice among D categories. Selecting these by arg max or by thresholding would make the mapping from logits to outputs piecewise constant, with zero gradient almost everywhere. The Gumbel-Softmax trick [Jang et al.(2017)] replaces this hard sampling step with a differentiable relaxation. For unnormalised log-probabilities $\pi_1, \ldots, \pi_K$, a Gumbel-Softmax sample is defined as

$$y_i = \frac{\exp((\pi_i + g_i)/\tau)}{\sum_{j=1}^{K} \exp((\pi_j + g_j)/\tau)}, \quad g_i = -\log(-\log(u_i)), \quad u_i \sim \text{Uniform}(0,1). \tag{15}$$

The Gumbel samples g_i carry all of the stochasticity in this expression; the logits π_i enter only through deterministic operations. Gradients with respect to π_i therefore propagate through Equation (15). The temperature $\tau > 0$ controls sharpness: as $\tau \to 0$ the softmax approaches a one-hot vector, and as $\tau \to \infty$ it approaches the uniform distribution.

Straight-through estimator. The classifier used to guide the generator was trained on discrete graphs with $\{0, 1\}$ adjacency and one-hot node features, and it is not meaningful to feed it continuous Gumbel-Softmax outputs at training time. The straight-through estimator resolves this by combining a hard forward pass with a soft backward pass: the hard one-hot sample $\tilde{y}_{\text{hard}} = \text{onehot}(\arg\max_i y_i)$ is used in the forward computation, while the gradient is taken with respect to the continuous sample $\tilde{y}_{\text{soft}}$ from Equation (15). The standard implementation uses the identity

$$\tilde{y} = \tilde{y}_{\text{hard}} + \tilde{y}_{\text{soft}} - \text{stopgrad}(\tilde{y}_{\text{soft}}),$$

where $\text{stopgrad}(\cdot)$ instructs autograd to treat its argument as a constant, so it contributes zero to any derivative. The value of $\tilde{y}$ is simply $\tilde{y}_{\text{hard}}$, since $\tilde{y}_{\text{soft}}$ and $\text{stopgrad}(\tilde{y}_{\text{soft}})$ take the same numerical value and cancel. Differentiating with respect to the generator parameters θ,

$$\frac{\partial \tilde{y}}{\partial \theta} = \underbrace{\frac{\partial \tilde{y}_{\text{hard}}}{\partial \theta}}_{=0} + \frac{\partial \tilde{y}_{\text{soft}}}{\partial \theta} - \underbrace{\frac{\partial\, \text{stopgrad}(\tilde{y}_{\text{soft}})}{\partial \theta}}_{=0} = \frac{\partial \tilde{y}_{\text{soft}}}{\partial \theta}.$$

The $\arg\max$ in $\tilde{y}_{\text{hard}}$ has zero gradient almost everywhere, and the stop-gradient term contributes zero by construction, so only the soft Gumbel-Softmax sample passes gradient to θ. The classifier therefore sees a discrete graph on the forward pass and the generator receives the Gumbel-Softmax gradient on the backward pass.

Application to the two heads. Each potential edge (i, j) is a binary choice between edge and no-edge, so 2-class logits $[\tilde{A}_{\text{raw}}[i, j],\ -\tilde{A}_{\text{raw}}[i, j]]$ are formed and passed through the Gumbel-Softmax with $K = 2$. Symmetry of the output adjacency is enforced by sampling only the upper triangle and mirroring it to the lower triangle:

$$\tilde{A}_{\text{upper}} = \text{triu}(\text{GumbelSoftmax}(\tilde{A}_{\text{raw}}, \tau)), \quad \tilde{A} = \tilde{A}_{\text{upper}} + \tilde{A}_{\text{upper}}^{\top}. \tag{16}$$

For node features, each node selects one of D types (for example, seven atom types on MUTAG), so the Gumbel-Softmax is applied with $K = D$ per node to obtain one-hot feature rows:

$$\tilde{X}[v, :] = \text{GumbelSoftmax}(\tilde{X}_{\text{raw}}[v, :],\ \tau) \in \{0, 1\}^D. \tag{17}$$

The temperature is $\tau = 1.0$ during training, which leaves the relaxation smooth enough to give useful gradients, and $\tau = 0.1$ during evaluation, which produces sharper, effectively discrete outputs.

3.4 Training Objective

The total generator loss combines adversarial, GNN-guidance, and structural regularisation terms:

$$\mathcal{L}_G = (1 - \lambda(t))\, \mathcal{L}_{\text{GAN}} + \lambda(t)\, \mathcal{L}_{\text{GNN}} + \mathcal{L}_{\text{reg}}, \tag{18}$$

where $\lambda(t) \in [0, 1]$ is a dynamic weight that increases over training, and $\mathcal{L}_{\text{reg}}$ comprises structural regularisation losses described below. The three terms correspond to three

goals: making the generated graphs look realistic ($\mathcal{L}_{\text{GAN}}$), making them belong to the target class ($\mathcal{L}_{\text{GNN}}$), and keeping their size, composition, and average degree close to the target class ($\mathcal{L}_{\text{reg}}$). We now explain each in turn, beginning with the adversarial component and the discriminator that drives it.

3.4.1 $\mathcal{L}_{\text{GAN}}$: Adversarial Loss

The adversarial component drives generated graphs toward the real-graph distribution. A discriminator $\mathcal{D}$, a small graph neural network trained from scratch alongside the generator, maps each graph to an unbounded scalar realism score: higher means more real. The discriminator is distinct from the frozen pretrained classifier Φ used in $\mathcal{L}_{\text{GNN}}$, since Φ scores class membership rather than realism. Generator and discriminator update in alternating steps with $n_{\text{critic}} = 1$ so that neither is trained to convergence; this co-evolution keeps $\mathcal{D}$'s scores informative across the region of input space where the generator's current outputs lie. We use the WGAN-GP formulation [Gulrajani et al.(2017)].

The generator interacts with the discriminator only through its own adversarial loss, the negation of the average score $\mathcal{D}$ assigns to generated graphs:

$$\mathcal{L}_{\text{GAN}} = -\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]. \tag{19}$$

Minimising $\mathcal{L}_{\text{GAN}}$ is equivalent to maximising the score $\mathcal{D}$ assigns to generated graphs. Real graphs do not appear in $\mathcal{L}_{\text{GAN}}$ directly; the generator learns about the real distribution only through $\mathcal{D}$'s gradient. The discriminator architecture, its training loss with the gradient penalty, the two-player training dynamics, and the derivation of why WGAN-GP avoids the failure modes of vanilla GANs are in Appendix A.

3.4.2 $\mathcal{L}_{\text{GNN}}$: GNN Guidance Loss

The adversarial loss enforces structural realism but does not steer generation toward any particular class. The GNN guidance loss maximises the pretrained classifier's confidence on the target class c:

$$\mathcal{L}_{\text{GNN}} = -\log p_{\Phi}(y = c \mid \tilde{G}), \tag{20}$$

where $p_{\Phi}(y = c \mid \tilde{G})$ is the softmax probability that the pretrained k-GNN Φ assigns to class c for the generated graph $\tilde{G}$. This is the negative log-likelihood of the target class. The classifier's weights are frozen; only the generator's weights are updated. Minimising $\mathcal{L}_{\text{GNN}}$ pushes the generator to produce graphs that the classifier assigns to class c.

3.4.3 $\lambda(t)$: Dynamic Weighting

The balance parameter $\lambda(t)$ in Equation (18) follows a sigmoid schedule [Sun et al.(2025)]. The general form in Sun et al. is an affine map from a sigmoid output into an arbitrary range $[\lambda_{\min}, \lambda_{\max}]$; the full range $[0, 1]$ is used here, so the floor and ceiling drop out and the schedule reduces to

$$\lambda(t) = \sigma\left(k \cdot \left(\frac{2(t/T - p)}{1 - p} - 1\right)\right), \tag{21}$$

where T is the total number of training iterations ($T = 300$ epochs in our experiments), $p = 0.4$ is the fraction of training before λ begins increasing, $k = 10$ controls the steepness of the transition, and $\sigma(x) = 1/(1 + e^{-x})$ is the logistic sigmoid.

When $t \ll pT$ (the first $\sim$120 epochs), the sigmoid argument is strongly negative, giving $\lambda(t) \approx 0$: the total loss is $\mathcal{L}_G \approx \mathcal{L}_{\text{GAN}}$, and the generator focuses on producing structurally realistic graphs without class-specific pressure. As t passes pT, $\lambda(t)$ increases toward 1: the total loss shifts to $\mathcal{L}_G \approx \mathcal{L}_{\text{GNN}}$, directing the generator toward class-specific patterns. Without this schedule, the generator collapses to structurally degenerate graphs that satisfy the classifier but bear no resemblance to real data, since $\mathcal{L}_{\text{GNN}}$ can be minimised by any graph that triggers the correct class output, regardless of structural plausibility.

3.4.4 $\mathcal{L}_{\text{reg}}$: Structural Regularisation

The generator outputs fixed-size $N \times N$ adjacency and $N \times D$ feature matrices ($N = 28$ for MUTAG, $N = 50$ for PROTEINS). Without explicit constraints, Gumbel-Softmax sampling activates most slots and settles on atom-type distributions that satisfy the classifier but do not resemble real class members. $\mathcal{L}_{\text{GAN}}$ constrains these statistics only indirectly through the discriminator and is too weak a signal to prevent these failures. $\mathcal{L}_{\text{reg}}$ therefore supplies direct supervision on three graph-level statistics: average degree, graph size, and atom-type frequencies. The class targets for all three are precomputed once from the real training set:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{degree}} \cdot \left(\frac{\hat{d} - \mu_c^d}{\sigma_c^d} \right)^2 + \lambda_{\text{size}} \cdot \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2 + \lambda_{\text{type}} \cdot \sum_{i=1}^{D} (\hat{p}_i - p_i^c)^2 , \tag{22}$$

with $\lambda_{\text{degree}} = 1$, $\lambda_{\text{size}} = 10$, and $\lambda_{\text{type}} = 1$. The first term penalises deviation from the class mean of average degree μ_c^d, scaled by the class standard deviation σ_c^d. The second term penalises deviation from the class mean graph size $\bar{n}_c$, where $\hat{n}$ is a soft count of active slots in the generated adjacency. The third term penalises deviation from the class atom-type distribution p^c, where $\hat{p}_i$ is the share of slots that the generator's near-one-hot feature output assigns to atom type i. The targets μ_c^d, σ_c^d, $\bar{n}_c$, and p^c are computed once from the real class-c training graphs.

Three design choices follow from the structure of the terms. The degree penalty is normalised by σ_c^d, so classes with broader degree variation get a broader target. The size penalty uses relative rather than absolute error, so λ_{size} does not need retuning across datasets with different mean graph sizes. The atom-type penalty uses squared L2 rather than KL divergence, so the loss remains well-defined when some atom types have $p_i^c = 0$, which occurs on MUTAG. $\mathcal{L}_{\text{reg}}$ has the shortest gradient path of the three loss components, since its gradients flow directly through the Gumbel-Softmax backward pass into the generator MLP without invoking the discriminator or the classifier. The definition of $\hat{d}$, the construction of $\hat{n}$ and $\hat{p}$, the choice of the sigmoid scale in the soft size count, and the full L2-vs-KL discussion are in Appendix B.

3.5 Dense Wrapper for Differentiable k-GNN Inference

The pretrained classifier cannot be used directly inside the generator's training loop. This section explains why, and how we fix it. The 1-GNN dense wrapper is taken from GIN-

Graph [Sun et al.(2025)]; the same logic is applied to the 2-GNN branch of the 1-2-GNN classifier in Section 3.5.4.

3.5.1 Why the sparse form breaks the gradient

The classifier was trained using PyTorch Geometric's sparse graph representation: the adjacency is stored as an `edge_index` tensor of integer source–destination pairs, and message passing is implemented as a `scatter_add` over edges rather than as a dense matrix product $\mathbf{AH}$. On discrete $\{0, 1\}$ inputs the two forms compute identical outputs; sparse is simply faster and more memory-efficient because it skips the $O(N^2)$ zero-multiplications that dominate $\mathbf{AH}$ when $\mathbf{A}$ is sparse.

This is fine for classifier training, where $\mathbf{A}$ is a fixed discrete input and no gradients need to flow through it. It breaks inside the generator's training loop, where the adjacency $\tilde{A} \in [0, 1]^{N \times N}$ is a continuous tensor produced by the Gumbel-Softmax straight-through estimator and $\nabla_\phi \mathcal{L}_{\text{GNN}}$ requires the derivative $\partial f_\Phi / \partial \tilde{A}$. The sparse forward has no slot for a float adjacency: converting $\tilde{A}$ into an `edge_index` requires a thresholding step whose derivative is zero everywhere. The chain rule for $\nabla_\phi \mathcal{L}_{\text{GNN}}$ then contains a zero factor, and the generator receives no signal from this term.

3.5.2 The fix

The classifier's forward pass is re-implemented using dense matrix operations. The pretrained weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}, \mathbf{W}_3^{(\ell)}, \mathbf{W}_4^{(\ell)}$ are copied from the sparse model unchanged and kept frozen throughout GIN-Graph training. Where the sparse version uses `scatter_add`, the dense version uses `torch.matmul` with $\tilde{A}$ as a continuous input tensor. Autograd differentiates matrix multiplication: for $\mathbf{Y} = \tilde{A}\,\mathbf{H}$, each entry $\partial \mathbf{Y} / \partial \tilde{A}_{ij} = \mathbf{H}[j, :]$ is well-defined. The chain rule no longer contains a zero factor, and $\nabla_\phi \mathcal{L}_{\text{GNN}}$ flows back to the generator.

The wrapper is not a different model. It is the same mathematical function with a forward that accepts continuous adjacency matrices. The rewrite is direct for the 1-GNN (Section 3.5.3, following [Sun et al.(2025)]). The 1-2-GNN extension (Section 3.5.4) is this work's contribution and is treated separately.

3.5.3 Dense 1-GNN

For the 1-GNN branch, the rewrite is direct. The node-level update (Equation 4) translates to dense batched computation. For a batch of $B = 64$ graphs with node features $\mathbf{X} \in \mathbb{R}^{B \times N \times d}$ and adjacency $\mathbf{A} \in [0, 1]^{B \times N \times N}$:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)}\mathbf{W}_1^{(t)} + \mathbf{A}\,\mathbf{H}^{(t-1)}\mathbf{W}_2^{(t)}\Big), \tag{23}$$

where the batch dimension is broadcast. The matrix product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ replaces `scatter_add`. Each entry $\tilde{A}_{ij}$ participates directly in the multiplication, so autograd can compute $\partial \mathbf{H}^{(t)} / \partial \tilde{A}_{ij}$. The weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}$ are copied from the pretrained sparse model and kept frozen. This step is taken directly from GIN-Graph [Sun et al.(2025)] and similar dense rewrites for differentiable graph generation. It is stated here to contrast with the 2-GNN case.

3.5.4 Dense 2-GNN

The same sparse-to-dense logic is applied to the 2-GNN branch. The construction is more involved than for the 1-GNN because the 2-set neighbourhood depends on adjacency lookups, which must also be made continuous.

Recall from Section 3.2 that the 2-GNN operates on node pairs $\{i, j\}$, and the neighbourhood definition (Equation 8) requires checking whether a candidate node w is adjacent in the original graph: $A_{iw} = 1$ when replacing i, or $A_{jw} = 1$ when replacing j. In the sparse implementation, this adjacency check uses a CSR (compressed sparse row) lookup: given node i, iterate over its edge list to find all w with $(i, w) \in E$. This is fast for discrete graphs, but it is the same type of discrete operation that blocks gradients, since the neighbour set is a list of integer indices with no derivative connecting it to $\tilde{A}$.

In the dense rewrite, this binary check becomes a continuous weight. Instead of checking whether w is a neighbour of i (answer: 0 or 1), we use $\tilde{A}_{iw} \in [0, 1]$ directly. Candidate pair features $\mathbf{g}[j, w]$ are multiplied by $\tilde{A}_{iw}$, contributing to the aggregation in proportion to $\tilde{A}_{iw}$. When $\tilde{A}_{iw} \approx 0$, the contribution is zero; when $\tilde{A}_{iw} \approx 1$, it is full; intermediate values contribute proportionally. On discrete $\{0, 1\}$ inputs, this reproduces the original neighbourhood rule exactly.

First, we construct pair features from the 1-GNN node embeddings $\mathbf{H}^{(T_1)} \in \mathbb{R}^{B\times N\times d_T}$. For each pair (i, j), we concatenate the two node embeddings in canonical min–max order with the continuous adjacency value $\tilde{A}_{ij}$:

$$\mathbf{F}[i, j] = \left[\mathbf{h}_{\min(i,j)} \parallel \mathbf{h}_{\max(i,j)} \parallel A_{ij}\right] \in \mathbb{R}^{2d_T+1}, \tag{24}$$

A 2-set $\{i, j\}$ is unordered, but the pair tensor $\mathbf{F}$ is indexed by the ordered tuple (i, j), so each pair occupies two tensor slots: $\mathbf{F}[i, j]$ and $\mathbf{F}[j, i]$. Without canonicalisation, these two slots would hold the same node embeddings in opposite concatenation order, $\mathbf{h}_i \parallel \mathbf{h}_j$ versus $\mathbf{h}_j \parallel \mathbf{h}_i$, and the downstream learnable transformation $\mathbf{W}_4^{(\ell)}$ would map them to different outputs, breaking the unordered-pair semantics of Equation (8). Sorting the indices by min and max enforces $\mathbf{F}[i, j] = \mathbf{F}[j, i]$ by construction, while preserving both node embeddings as distinct blocks so that the pair-level MLP can still distinguish their roles. The third entry $\tilde{A}_{ij}$ is the isomorphism-type indicator; in the sparse model this was the hard adjacency entry $A_{ij} \in \{0, 1\}$, whereas here it is the continuous value from the generator. It is already symmetric because $\tilde{A}$ is symmetrised at the sampling stage (Equation (16)) and requires no further canonicalisation.

For pair-level message passing, recall from Equation (8) that each 2-set $\{i, j\}$ has two kinds of neighbours: pairs formed by replacing i with some w adjacent to i, and pairs formed by replacing j with some w adjacent to j. In the dense version, "adjacent to i" is the continuous weight $\tilde{A}_{iw}$. Using the transformed features $\mathbf{g} = \mathbf{F}\,\mathbf{W}_4^{(\ell)} \in \mathbb{R}^{B\times N\times N\times d_T}$, the aggregation for pair (i, j) sums over all candidate w, weighted by $\tilde{A}$:

$$\mathrm{agg}_1[i, j] = \sum_{\substack{w=1 \\ w\neq j}}^{N} A_{iw} \cdot \mathbf{g}[j, w] \qquad \text{(replace } i \text{ with } w \text{ adjacent to } i\text{)}, \tag{25}$$

$$\mathrm{agg}_2[i, j] = \sum_{\substack{w=1 \\ w\neq i}}^{N} A_{jw} \cdot \mathbf{g}[i, w] \qquad \text{(replace } j \text{ with } w \text{ adjacent to } j\text{)}. \tag{26}$$

The exclusions $w \neq j$ in agg_1 and $w \neq i$ in agg_2 prevent including the self-pair $\{j, j\}$ or $\{i, i\}$, which is not a valid 2-set. The self-loop exclusion ($A_{ii} = 0$) removes $w = i$ from agg_1 and $w = j$ from agg_2, but the terms $w = j$ in agg_1 and $w = i$ in agg_2 must be explicitly masked. We zero the diagonal of $\mathbf{g}$ (setting $\mathbf{g}[k, k] = \mathbf{0}$ for all k) before summation, which eliminates both boundary terms: $\mathbf{g}[j, j] = \mathbf{0}$ removes the $w = j$ term in agg_1, and $\mathbf{g}[i, i] = \mathbf{0}$ removes the $w = i$ term in agg_2.

These sums are computed via Einstein summation (`einsum`) over the masked $\mathbf{g}$. The full 2-GNN layer update is:

$$\mathbf{F}^{(\ell+1)} = \sigma\Big(\mathbf{F}^{(\ell)}\,\mathbf{W}_3^{(\ell)} + \text{agg}_1^{(\ell)} + \text{agg}_2^{(\ell)}\Big). \tag{27}$$

After $T_2 = 2$ layers, the 2-GNN readout sums over the upper-triangular entries (one entry per canonical pair):

$$\mathbf{h}_G^{(2)} = \sum_{i<j} \mathbf{F}^{(T_2)}[i, j]. \tag{28}$$

3.5.5 Summary

The dense wrapper preserves the exact mathematical behaviour of the pretrained classifier on discrete graphs while extending its domain to continuous adjacency matrices. It is an implementation change, not a model change: all pretrained weights are reused unchanged. For the 1-GNN branch, the change is replacing `scatter_add` with `matmul`, as in GIN-Graph [Sun et al.(2025)]. For the 2-GNN branch, the discrete adjacency check $A_{iw} \in \{0, 1\}$ is replaced by a continuous weight $\tilde{A}_{iw} \in [0, 1]$ that participates in the computation graph. Without this, the GIN-Graph training loop, which requires $\partial f_\Phi / \partial \tilde{A}$ to be well-defined, cannot be run against a 1-2-GNN classifier.

3.6 Validation Score

At evaluation time, both the generator and the classifier are frozen; no gradients are computed. We sample 100 graphs per class at temperature $\tau = 0.1$ (which produces sharper, more discrete outputs than the training temperature of $\tau = 1.0$) and assign each graph a score that quantifies its quality as an explanation. The score must be low if the graph fails on any of three axes (class identity, internal representation, or structural plausibility), so that success on one axis cannot compensate for failure on another.

The composite validation score [Sun et al.(2025)] is:

$$v = (s \cdot p \cdot d)^{1/3}, \tag{29}$$

where p measures the classifier's prediction, s measures where the graph sits in the classifier's internal embedding space, and d measures whether the graph has a realistic amount of connectivity.

The geometric mean enforces non-compensation between components. If a generated graph has $p = 0.95$, $s = 0.95$, but $d = 0.05$ (correct class, correct embedding region, wrong connectivity), the geometric mean gives $v \approx 0.36$, correctly indicating failure. An arithmetic mean would give $v \approx 0.65$, above the validity threshold, hiding the structural problem. We now define each component.

3.6.1 Prediction Probability (p)

The prediction probability is the frozen classifier's softmax output for the target class:

$$p = p_\Phi(y = c \mid \tilde{G}).$$

This is the same quantity the generator was trained to maximise via $\mathcal{L}_{\text{GNN}}$ (Equation 20). For a well-trained generator, p is close to 1 at evaluation time. This is expected: the generator spent hundreds of epochs optimising this quantity. The component p therefore functions as a sanity check that the generator converged, not as a discriminator between good and poor generators. A generator producing graphs with $p < 0.5$ did not converge.

3.6.2 Embedding Similarity (s)

For any graph G the classifier processes, the forward pass produces a single vector $\mathbf{h}_G$ representing the entire graph. In the 1-GNN, this vector is the sum pool of the final node embeddings: $\mathbf{h}_G = \sum_{v \in V} \mathbf{h}_v^{(T_1)} \in \mathbb{R}^{d_T}$ (Equation 5). In the 1-2-GNN, it is the concatenation of the node-level and pair-level readouts: $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \| \mathbf{h}_G^{(2)}] \in \mathbb{R}^{2d_T}$ (Equation 10). One vector per graph, in the classifier's embedding space. Every forward pass on any graph produces exactly one such vector.

The *class centroid* $\boldsymbol{\mu}_c$ is the mean of $\mathbf{h}_G$ over all real training graphs in class c:

$$\boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G.$$

For MUTAG class 1 (Mutagen), $\mathcal{G}_c$ contains 125 molecules, so $\boldsymbol{\mu}_c$ is the average of 125 embedding vectors. The centroid sits in the same space as any individual $\mathbf{h}_G$ and marks the average location where real class-c graphs lie according to the classifier. It is computed once using the classifier's sparse implementation (real graphs are discrete, so sparse is appropriate), stored in the GIN-Graph checkpoint, and reused for every evaluation.

The embedding similarity is then the cosine similarity between the generated graph's embedding and this centroid:

$$s = \cos(\mathbf{h}_{\tilde{G}}, \boldsymbol{\mu}_c) = \frac{\mathbf{h}_{\tilde{G}} \cdot \boldsymbol{\mu}_c}{\|\mathbf{h}_{\tilde{G}}\| \, \|\boldsymbol{\mu}_c\|}, \quad \text{where } \boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G. \tag{30}$$

Cosine similarity measures the angle between two vectors, normalised so that identical direction gives $s = 1$ and orthogonal vectors give $s = 0$. A high s means the generated graph occupies the same region of embedding space as real class-c graphs. A low s means the classifier assigns the graph to class c but via an internal representation far from typical class members; the generator has found an input that triggers the correct output without resembling the real data in the classifier's internal representation.

Note on the computation of s. The GIN-Graph reference implementation [Sun et al.(2025)] uses the prediction probability p as a proxy for embedding similarity, setting $s = p$ and reducing the validation score to $v = (p^2 \cdot d)^{1/3}$. In this work, s is computed as the actual cosine similarity between $\mathbf{h}_{\tilde{G}}$ and $\boldsymbol{\mu}_c$, using the dense wrapper's `get_embedding()` method to extract the embedding from the frozen classifier for a continuous-valued generated graph. On PROTEINS Non-Enzyme, this distinction is visible in the results: both models achieve high p (0.919 and 0.998) but low s (0.356 and 0.151), a difference that would not appear under the $s = p$ proxy.

3.6.3 Degree Score (d)

The average degree of a graph is $\bar{d}_{\tilde{G}} = 2|\tilde{E}|/|\tilde{V}|$: total degree divided by node count, since each edge contributes 1 to each of its two endpoints. Let μ_d and σ_d be the mean and standard deviation of average degrees across real class-c graphs, both precomputed once from the training set. The degree score is:

$$d = \exp\left(-\frac{(\bar{d}_{\tilde{G}} - \mu_d)^2}{2\sigma_d^2}\right), \tag{31}$$

which is a Gaussian centred at the class mean μ_d with width σ_d. When $\bar{d}_{\tilde{G}} = \mu_d$, the score is $d = 1$. As the generated degree deviates from the mean, d decreases: one standard deviation off gives $d \approx 0.61$, two standard deviations give $d \approx 0.14$, three give $d \approx 0.01$. Classes with high variance in graph sizes get a wider (more permissive) Gaussian than classes with uniform sizes.

The degree score provides a structural sanity check that s and p cannot provide. Both s and p operate in the classifier's learned embedding space. A generator could produce graphs with correct class identity and correct embedding location but with edge counts far from any real graph. The degree score catches this failure mode. It does not capture fine-grained structural properties (motif frequencies, degree distributions), but it reliably rejects graphs with wrong connectivity at the coarsest level.

3.6.4 Validity Threshold

A generated graph is considered **valid** if two conditions hold: (i) $v > 0.5$, and (ii) $\bar{d}_{\tilde{G}}$ falls within 3 standard deviations of μ_d. The degree score d therefore appears twice: once as a soft component inside v, and once as a hard gate via the 3σ condition. The hard gate catches graphs with abnormal connectivity regardless of their s and p values. A graph with $\bar{d}_{\tilde{G}}$ far outside the class range is rejected even if it scores well on the other two components. Any validity rates discussed below are filtered on both conditions.

3.6.5 Evaluation Protocol

Part A uses 100 generated graphs per configuration to compute validity rates and report the mean validation score and component decomposition s, p, d (Table 3). Part B uses larger 1000-graph generated populations for the qualitative comparison, since population fingerprints such as atom-type shares, edge-pair shifts, and cycle counts are too noisy when read from a small number of samples. All quantities are forward-pass computations on frozen networks. No training or gradients are involved during evaluation.

4 Experimental Setup

4.1 Datasets

We evaluate on two standard graph classification benchmarks (Table 1).

MUTAG [Debnath et al.(1991)] contains 188 molecular graphs labelled by mutagenic effect on *Salmonella typhimurium*. Nodes represent atoms (C, N, O, F, I, Cl, Br) and

Table 1: Dataset statistics. GIN Gen is the maximum number of nodes used for explanation generation.

Dataset	Graphs	Node Feats	Max Nodes	GIN Gen	Task
MUTAG	188	7 (atom types)	28	28	Mutagenicity
PROTEINS	1113	3 (sec. str.)	620	50	Enzyme / non-enzyme

edges represent chemical bonds. The raw TU labels are $\{-1, +1\}$ with $+1$ denoting mutagenic compounds (125 graphs) and -1 non-mutagenic compounds (63 graphs). PyG's `TUDataset` remaps the labels by ascending sort, so in this work *Non-Mutagen* is class 0 (63 graphs) and *Mutagen* is class 1 (125 graphs).

PROTEINS [Borgwardt et al.(2005)] contains 1113 protein graphs where nodes are secondary structure elements (helix, sheet, coil/turn) connected if they are neighbours in the amino acid sequence or within a distance threshold in 3D space. Using the same ascending-sort remap, raw label 1 (enzymes, 663 graphs) maps to class 0 and raw label 2 (non-enzymes, 450 graphs) maps to class 1, so the classes are *Enzyme* (class 0) and *Non-Enzyme* (class 1). For GIN-Graph generation, we cap the graph size at 50 nodes (the median protein graph has $\sim$26 nodes), as the explanations highlight key substructures rather than reproducing entire large graphs.

4.2 Model Configuration

All k-GNN models use a hidden dimension of $d_T = 64$. The 1-GNN uses $T_1 = 3$ message-passing layers. The 1-2-GNN uses 3 layers for the 1-GNN component and $T_2 = 2$ layers for the 2-GNN component. Both use dropout of 0.5 and are trained with Adam (lr = 0.01) using cross-entropy loss. Table 2 reports the best epoch observed for each run. Data is split 80/20 into train/test with a fixed seed of 42.

The GIN-Graph generator uses a latent dimension $d_z = 32$, hidden dimension $d_h = 128$, and is trained for 300 epochs with Adam (lr = 0.001) and batch size 64. WGAN-GP uses $\lambda_{\text{GP}} = 10$ and trains the discriminator once per generator update ($n_{\text{critic}} = 1$). The dynamic weighting uses $p = 0.4$ and $k = 10$ (Equation 21). The generation size is fixed at $N = 28$ for MUTAG and $N = 50$ for PROTEINS.

4.3 Evaluation Protocol

For each model-dataset-class combination, we generate 100 explanation graphs at evaluation temperature $\tau = 0.1$ and evaluate them using the validation score (Equation 29). We compute the validity rate and report:

- **Mean validation score**: averaged over all 100 generated graphs;
- **Component scores**: embedding similarity (s), prediction probability (p), degree score (d).

5 Results

The goal of this section is to describe, without evaluative judgement, what each classifier has internalised as the class concept on each dataset. We do not have a ground-truth notion of a "correct" explanation, so claims are restricted to describing where the real data, the 1-GNN-generated data, and the 1-2-GNN-generated data differ, and where they coincide. We do not claim one architecture produces better explanations than the other.

Section 5.1 (Part A) first checks that all three components of the pipeline, namely the classifiers, the generators, and the validation score, are functioning before any comparison is interpreted. Section 5.2 (Part B) then performs the three-way qualitative comparison per (dataset, class) cell.

5.1 Part A: Pipeline sanity check

5.1.1 Classifier accuracy

Table 2 reports test-set accuracy for both classifiers on both datasets. On MUTAG, both achieve the same best test accuracy (89.5%); on PROTEINS, the 1-GNN reaches 79.8% and the 1-2-GNN 78.9%. Both classifiers therefore provide a usable target for GIN-Graph training. The two-point accuracy gap on PROTEINS is below the single-seed noise level discussed in Section 6.2, so we do not read it as evidence of architectural superiority.

Table 2: k-GNN classification results. Best test accuracy reported over all training epochs.

Dataset	Model	Params	Final Test Acc	Best Acc	Best Epoch
MUTAG	1-GNN	21,570	81.6%	89.5%	84
MUTAG	1-2-GNN	50,370	76.3%	89.5%	18
PROTEINS	1-GNN	21,058	75.8%	79.8%	120
PROTEINS	1-2-GNN	49,858	73.1%	78.9%	75

5.1.2 Generator training behaviour

All eight generator runs reached the 300-epoch horizon without divergence. The adversarial losses are not expected to decrease monotonically, because the generator and discriminator are trained against each other. The useful sanity check is instead that the training remains bounded while the target-class prediction probability rises and the dynamic weight schedule shifts emphasis from adversarial realism to class-specific guidance.

5.1.3 Validation-score component sanity

Table 3 reports the mean validation score v and its components (s, p, d) averaged over 100 graphs from each of the eight generators. Two observations confirm the components are working as intended:

- Prediction probability p is high ($p \geq 0.56$ across all cells, $p \geq 0.92$ in seven of eight) confirming that each generator has learned to target its trained class.

- Embedding similarity s and degree score d lie strictly inside $(0, 1)$ in every cell. They are not collapsed to 0 or 1, so the metric is informative rather than saturated.

The spread of s across cells (from 0.151 to 0.987) is the signal the qualitative comparison will interpret in Part B.

Table 3: Validation-score components per generator (mean over 100 generated graphs).

Dataset	**Class**	**Model**	v	s	p	d
MUTAG	0 (Non-Mutagen)	1-GNN	0.320	0.700	1.000	0.210
MUTAG	0 (Non-Mutagen)	1-2-GNN	0.434	0.771	1.000	0.314
MUTAG	1 (Mut.)	1-GNN	0.681	0.935	1.000	0.490
MUTAG	1 (Mut.)	1-2-GNN	0.735	0.933	1.000	0.565
PROTEINS	0 (Enz.)	1-GNN	0.986	0.987	0.986	0.984
PROTEINS	0 (Enz.)	1-2-GNN	0.760	0.782	0.564	0.996
PROTEINS	1 (Non-Enzyme)	1-GNN	0.330	0.356	0.919	0.837
PROTEINS	1 (Non-Enzyme)	1-2-GNN	0.524	0.151	0.998	0.961

5.2 Part B: Qualitative three-way comparison

This part compares three graph populations: the real dataset graphs, the graphs generated for the 1-GNN, and the graphs generated for the 1-2-GNN. The aim is not to decide that one model is better. The aim is to see what kind of graph each classifier treats as a typical example of each class.

The analysis is based mainly on population-level statistics. Individual graph drawings are useful for visual intuition, but they can be misleading if they are treated as evidence by themselves. The generator produces one graph from one random latent vector, so a few examples only show a small part of the generated distribution. Figures 2 and 6 are therefore included only to make the populations easier to picture. The claims below are based on 1000 generated graphs per dataset, class, and model. The thresholded adjacency is symmetric and zero on the diagonal in every case, and isolated nodes are dropped before per-graph statistics are computed. The remaining active graph is not forced to be connected, so the cycle fingerprints should be read as population-level structural fingerprints rather than as claims about one connected molecule or protein-like component per sample.

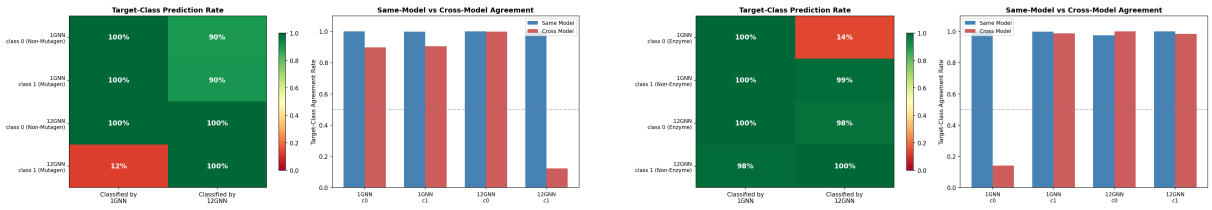
5.2.1 Prediction and cross-model agreement

The first check is whether the generated graphs are recognised by the model that generated them. Table 4 and Figure 1 report same-model and cross-model agreement. Same-model agreement measures whether the generating model classifies its own generated graphs as the target class. Cross-model agreement measures whether the other classifier also accepts those graphs as the same class.

Across almost all settings, the generated graphs are classified as the intended class by their own model with very high agreement. This means the generator is not just producing random graphs. It is producing graphs that the classifier itself accepts as class examples.

Table 4: Cross-model classification of generated graphs (1000 per cell).

Generated by	Target class	Same	Cross
MUTAG			
1-GNN	Non-Mutagen (c0)	100.0%	89.8%
1-GNN	Mutagen (c1)	99.9%	90.5%
1-2-GNN	Non-Mutagen (c0)	100.0%	99.9%
1-2-GNN	Mutagen (c1)	100.0%	12.3%
PROTEINS			
1-GNN	Enzyme (c0)	100.0%	14.0%
1-GNN	Non-Enzyme (c1)	99.9%	98.8%
1-2-GNN	Enzyme (c0)	97.6%	100.0%
1-2-GNN	Non-Enzyme (c1)	100.0%	98.5%



(a) MUTAG (b) PROTEINS

Figure 1: Cross-model classification of generated graphs. Each cell shows the fraction classified into the target class by each model, computed on 1000 generated graphs per generator-class cell.

The more interesting result is that this agreement does not always transfer to the other classifier.

Two cells stand out. On MUTAG class 1, the 1-2-GNN generated Mutagen graphs are accepted by the 1-2-GNN (100%), but mostly rejected by the 1-GNN (12.3% cross agreement). On PROTEINS class 0, the 1-GNN generated Enzyme graphs are accepted by the 1-GNN (100%), but mostly rejected by the 1-2-GNN (14.0% cross agreement). These two cases show that the generated graphs are classifier-specific. A graph can look like a good class example to one classifier while not being accepted by the other.

5.2.2 MUTAG

MUTAG is a molecule dataset, so the most direct comparison is based on atom types, graph size, edge types, and cycle structure. Table 5 summarises the real and generated populations. In both classes, the generated graphs keep the mean degree close to the real data. The larger differences appear in graph size and atom composition.

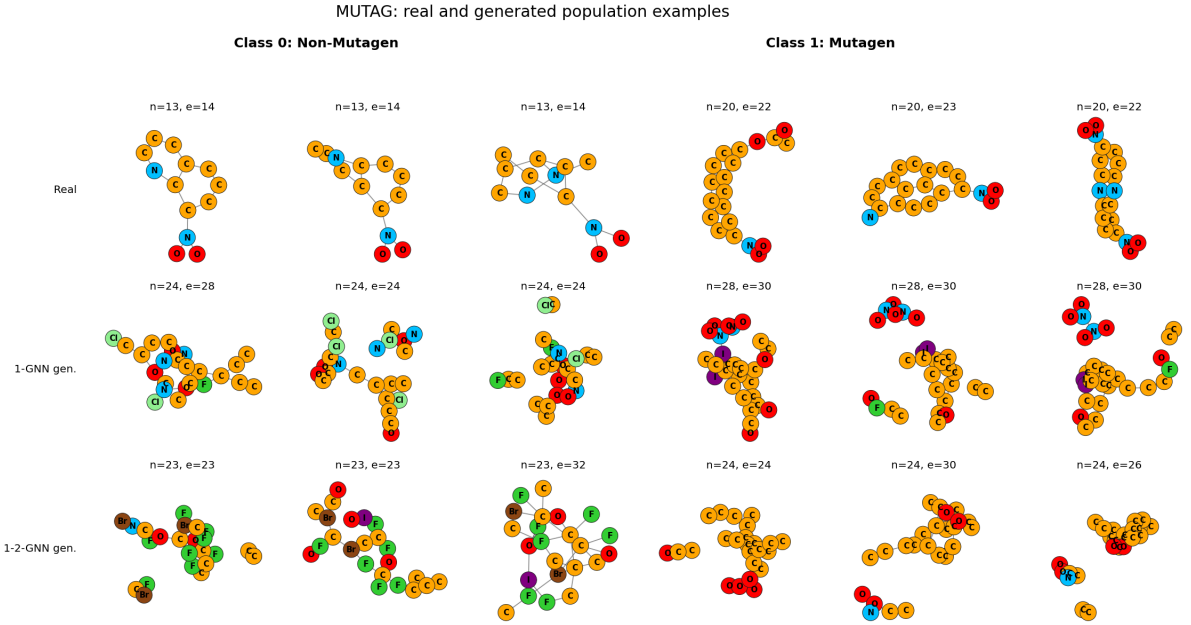


Figure 2: Illustrative MUTAG examples. Each row shows three median-sized examples from one source: real dataset graphs, the 1-GNN generated population, or the 1-2-GNN generated population. These examples are for visual orientation only; the quantitative claims use the full 1000-graph generated populations.

Atom-type distribution shift. Figure 3 shows the same atom-composition result as generated-minus-real percentage-point shifts. For class 0, the 1-2-GNN generator changes the atom distribution strongly. It produces many more halogen atoms, especially fluorine and bromine, and almost removes nitrogen. The 1-GNN also increases halogens compared with the real data, but the effect is much smaller. This suggests that the 1-2-GNN has learned a more extreme Non-Mutagen prototype for this class.

For class 1, the pattern is different. The real Mutagen class contains mostly carbon, oxygen, and nitrogen, with almost no halogens. The 1-GNN generator introduces rare

Table 5: MUTAG three-way comparison per class. Sizes, degree, and atom percentages aggregated over 1000 generated graphs per configuration; only the most populated atom types are shown.

Class	Source	$\bar{n}$	$\bar{d}$	%C	%N	%O	%halogen (F+Cl+Br+I)
0 (Non-Mutagen)	real	13.9	2.09	64.2	13.0	18.8	4.1
	1-GNN gen	23.8	2.07	57.1	11.1	17.0	14.9
	1-2-GNN gen	23.2	2.07	45.7	0.1	12.5	41.7
1 (Mutagen)	real	19.8	2.24	73.2	9.3	17.1	0.4
	1-GNN gen	27.3	2.23	63.4	7.5	19.1	10.0
	1-2-GNN gen	24.3	2.25	79.8	2.9	17.2	0.0

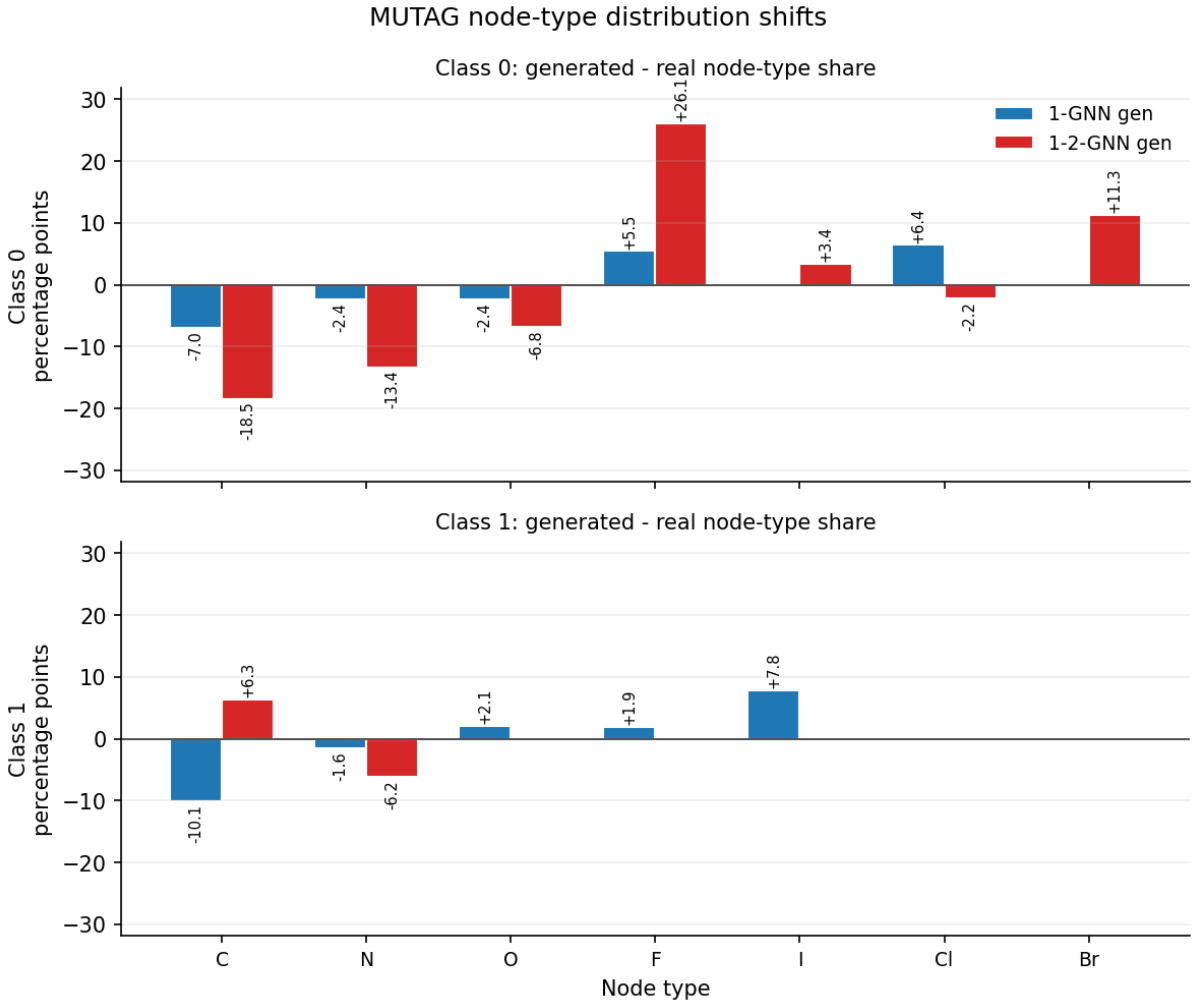


Figure 3: MUTAG atom-type distribution shifts. Bars show generated-minus-real changes in active-node atom shares, in percentage points, over 1000 generated graphs per class. Positive bars are over-produced by the generator; negative bars are under-produced.

halogens, especially iodine and fluorine, even though these are nearly absent in the real class. The 1-2-GNN generator does the opposite. It keeps halogens close to zero, increases carbon, and reduces nitrogen. This means the 1-2-GNN is not simply more halogen-heavy on MUTAG in general. The effect is class-specific.

Edge-pair distribution shift. The edge-pair shift plot supports the same reading. For MUTAG class 0, the 1-2-GNN creates many more C-F and C-Br edges than the real data. This means the atom shift also appears in the bond structure. For class 1, the largest positive shifts are C-O and O-O edges, while C-N and N-O edges are reduced. The Mutagen prototype is therefore not just low-halogen; it also changes which common atom pairs are connected.

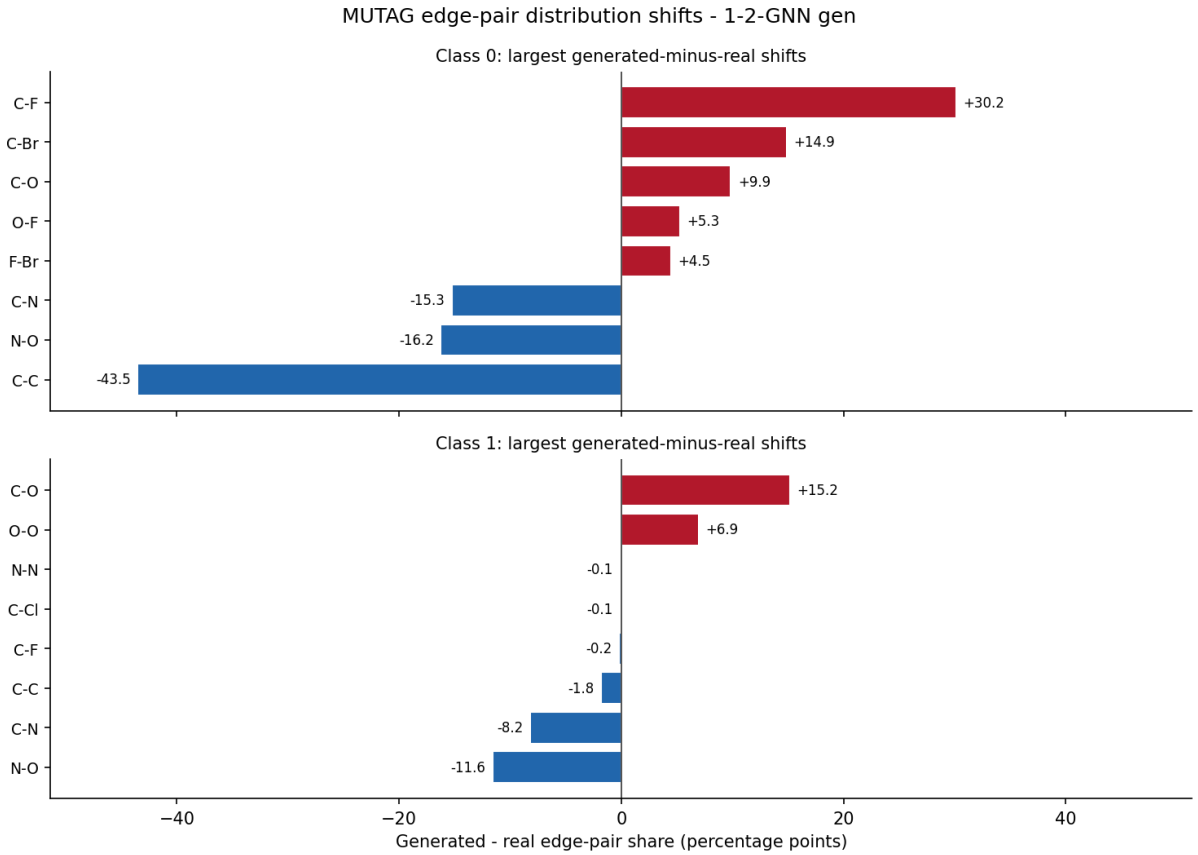


Figure 4: MUTAG edge-type co-occurrence fingerprint for the 1-2-GNN generator. Bars show the largest generated-minus-real changes in unordered atom-pair edge shares, in percentage points, over 1000 generated graphs per class. Positive bars are over-produced by the generator; negative bars are under-produced.

Cycle-count fingerprint. The cycle-count plot gives a structural comparison that is especially relevant for the difference between 1-WL and 2-WL. Real MUTAG molecules in these classes contain no 3-cycles or 4-cycles. Both generators introduce such cycles, so neither generated population is a faithful sample of the real molecular distribution. However, the 1-2-GNN introduces fewer triangles than the 1-GNN in both classes. This is most clear in class 1, where the 1-GNN has about one triangle per graph on average,

while the 1-2-GNN has far fewer. This fits the theory, since pair-level message passing can distinguish triangle-like structure more directly than a standard 1-GNN.

Table 6: MUTAG cycle counts (mean simple cycles per graph, exact). Real classes contain no triangles or 4-cycles; both generators introduce them, but the 1-2-GNN suppresses 3-cycles much more strongly than the 1-GNN.

Class	Source	3-cycles	4-cycles	5-cycles	6-cycles
0 (Non-Mutagen)	real	0.00	0.00	0.21	1.48
	1-GNN gen	0.82	0.96	1.22	1.46
	1-2-GNN gen	0.50	1.42	0.97	1.63
1 (Mutagen)	real	0.00	0.00	0.44	3.02
	1-GNN gen	1.01	1.19	3.31	3.47
	1-2-GNN gen	0.15	3.88	3.21	2.29

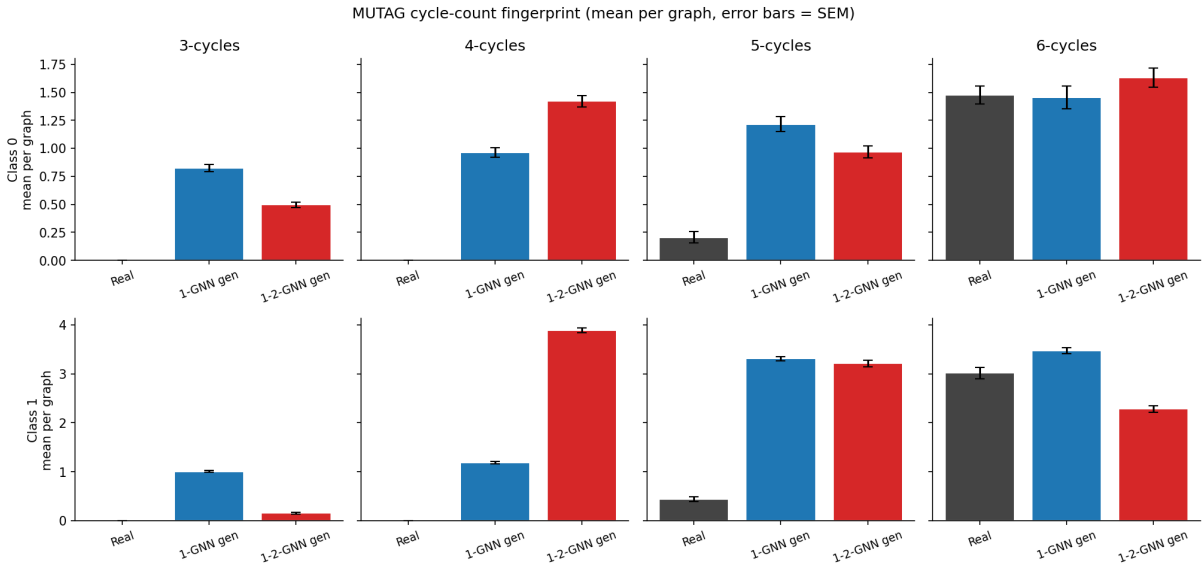


Figure 5: MUTAG simple-cycle counts per graph, aggregated over 1000 generated graphs per cell. Error bars are the standard error of the mean. Real MUTAG has zero 3- and 4-cycles in both classes; both generators introduce them, but the 1-2-GNN's 3-cycle rate is lower than the 1-GNN's, while it over-produces 4-cycles.

At the same time, the 1-2-GNN overproduces 4-cycles, especially in class 1. This is not a contradiction of the triangle result: the theory-aligned signal here is about triangle-like structure, not a guarantee that every short cycle will be penalised in the same direction. A careful interpretation is that the 1-2-GNN learns a different structural prototype, not a universally better one. The strongest MUTAG result is still the cross-model result for class 1: 1-2-GNN generated Mutagen graphs are accepted by the 1-2-GNN, but mostly rejected by the 1-GNN.

5.2.3 PROTEINS

PROTEINS has a different type of graph structure. The nodes are secondary-structure types, here written as H for Helix, S for Sheet, and C for Coil/Turn. The edges represent

structural or sequential relationships between these elements. Table 7 gives the population summary.

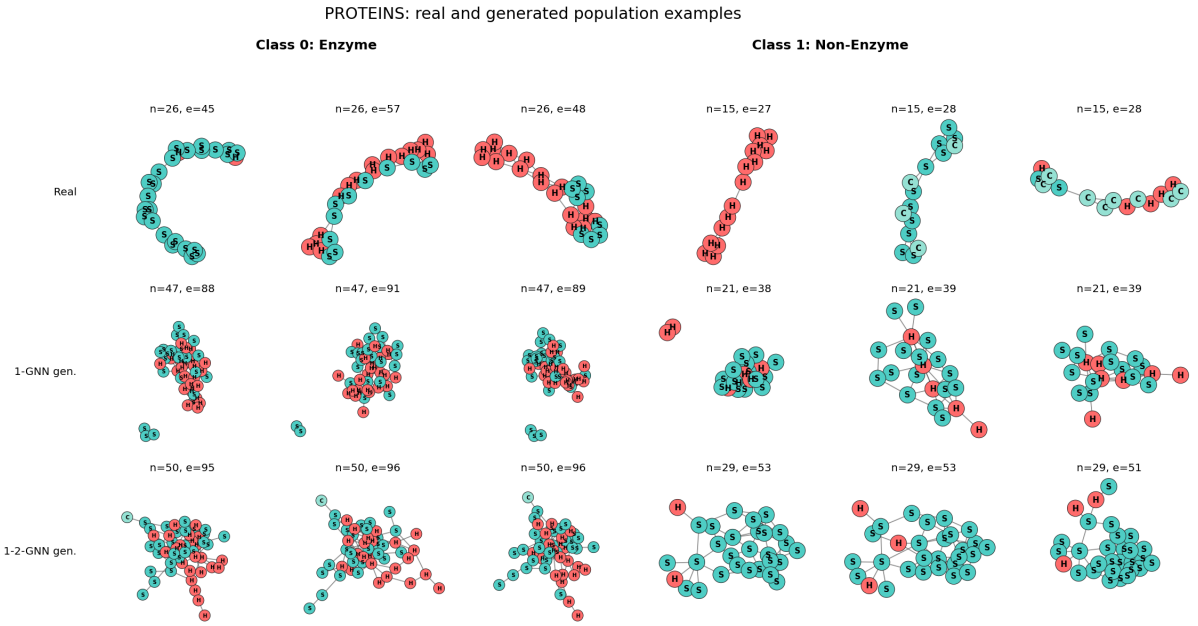


Figure 6: Illustrative PROTEINS examples. Each row shows three median-sized examples from one source: real dataset graphs, the 1-GNN generated population, or the 1-2-GNN generated population. The generated examples make the population-level shifts easier to see, but they are not used as standalone evidence.

Table 7: PROTEINS three-way comparison per class. Sizes, degree, and secondary-structure percentages aggregated over 1000 generated graphs per configuration.

Class	Source	$\bar{n}$	$\bar{d}$	%Helix	%Sheet	%Coil/Turn
0 (Enzyme)	real	48.7	3.80	50.4	46.5	3.1
	1-GNN gen	46.7	3.81	54.2	45.8	0.0
	1-2-GNN gen	50.0	3.80	50.9	47.1	2.0
1 (Non-Enzyme)	real	23.7	3.64	40.7	56.2	3.1
	1-GNN gen	24.1	3.72	24.8	75.2	0.0
	1-2-GNN gen	29.6	3.64	11.3	88.7	0.0

Secondary-structure distribution shift. For PROTEINS class 0, the node-type distribution is fairly close to the real data, especially for the 1-2-GNN. The real class has a mix of Helix and Sheet nodes, with a small amount of Coil/Turn. The 1-2-GNN generated graphs keep this balance quite well. The 1-GNN is also close, although it removes Coil/Turn almost completely. Both generators keep mean degree close to the real class.

This should be read together with the cross-classification result: 1-GNN Enzyme graphs are mostly rejected by the 1-2-GNN, while 1-2-GNN Enzyme graphs are accepted by both models.

For PROTEINS class 1, the main pattern is a strong shift toward Sheet nodes. Both generators overproduce Sheet and underproduce Helix and Coil/Turn, but the 1-2-GNN

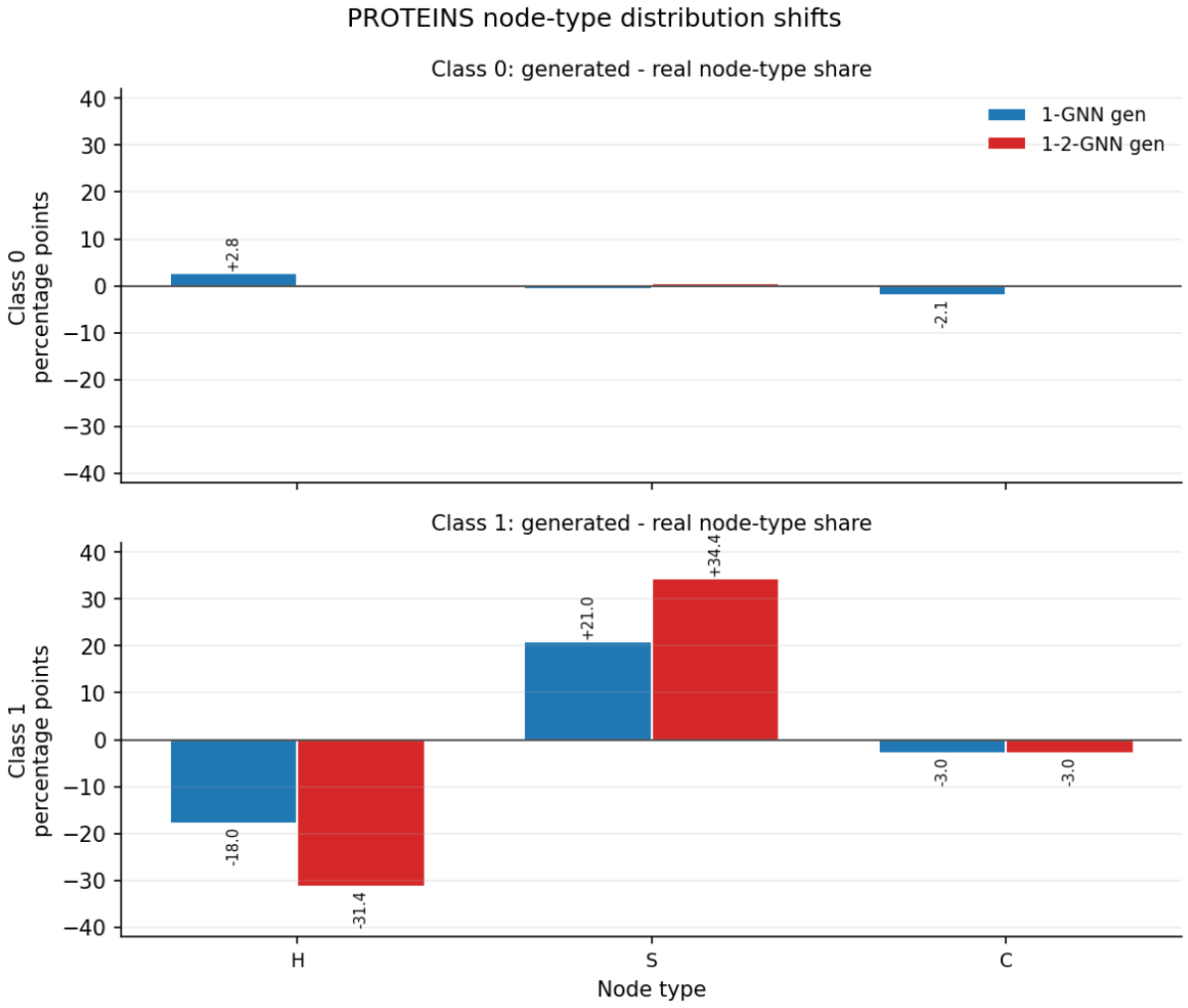


Figure 7: PROTEINS secondary-structure distribution shifts. Bars show generated-minus-real changes in active-node type shares, in percentage points, over 1000 generated graphs per class. Here H denotes Helix, S denotes Sheet, and C denotes Coil/Turn.

does this more strongly. The 1-GNN generated graphs are already Sheet-heavy, but the 1-2-GNN generated graphs are more extreme. This is a clear example of overcompensation: the generator seems to have learned that Sheet structure is useful for the class, then produces too much of it compared with the real data.

Edge-pair distribution shift. The edge-pair plot adds another layer. In PROTEINS class 0, the 1-2-GNN keeps the node-type distribution close to real data, but still shifts edges toward mixed H-S pairs and away from S-S pairs. In class 1, the change is much larger: the 1-2-GNN strongly overproduces S-S edges and underproduces H-H and H-S edges. This matters because the generated graphs are not only changing the number of Sheet nodes. They are also changing how those nodes are connected.

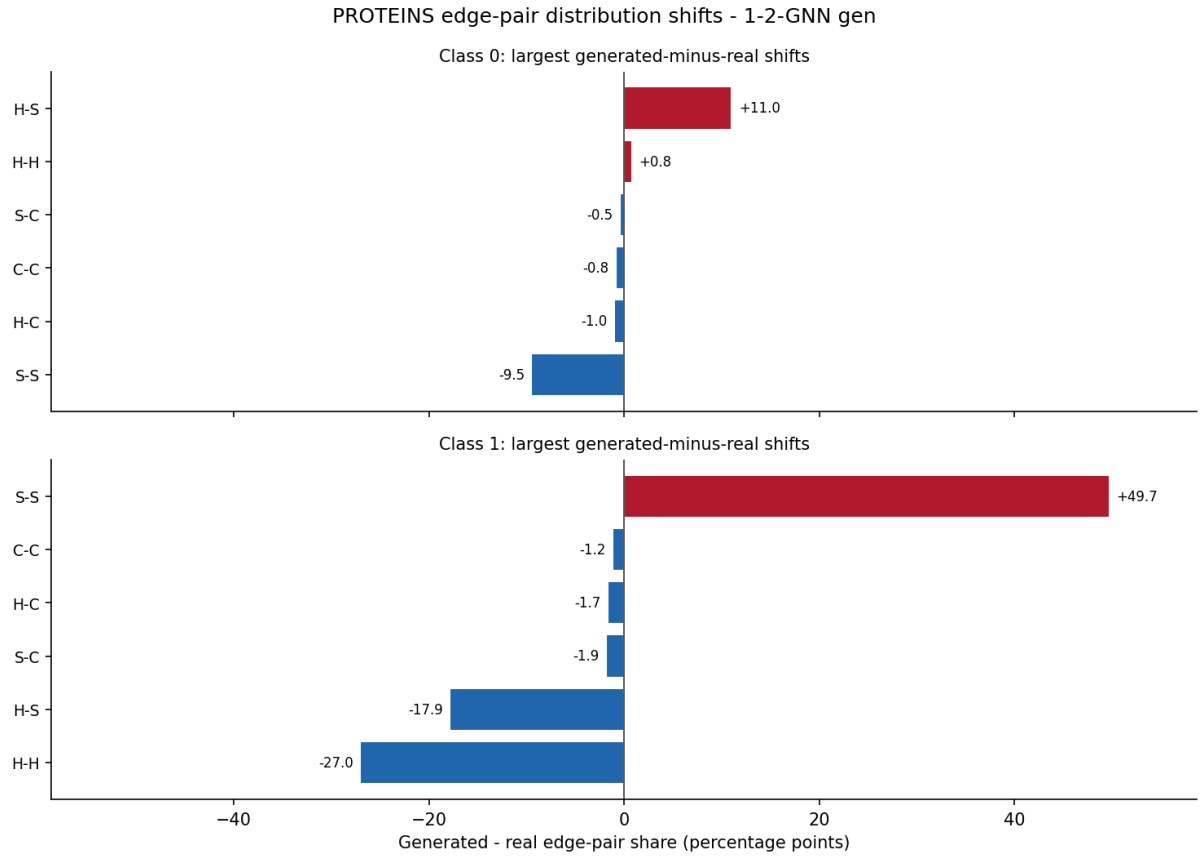


Figure 8: PROTEINS edge-type co-occurrence fingerprint for the 1-2-GNN generator. Bars show the largest generated-minus-real changes in unordered secondary-structure edge-pair shares, in percentage points, over 1000 generated graphs per class. Positive bars are over-produced by the generator; negative bars are under-produced.

Cycle-count fingerprint. The cycle plots show another kind of structural drift. In PROTEINS, both generators overproduce 5- and 6-cycles compared with the real data. This should not be explained simply as the generator producing more edges: the mean degree of the generated graphs is close to the real mean degree in most PROTEINS settings, and the normalized cycle view in Figure 10 shows the same broad pattern after dividing by graph size or edge count. A better explanation is that the generators arrange a similar amount of local connectivity into more cyclic structures.

The direction depends on the class. In class 0, the 1-2-GNN has the largest 6-cycle excess, including after normalization. This is a useful argument for architecture-specific structure: the pair-level model is not just matching the Helix/Sheet proportions, but forming larger cyclic closures in a way the 1-GNN does not. In class 1, the ordering reverses: the 1-GNN is more cycle-heavy than the 1-2-GNN, even though the 1-2-GNN is more Sheet-heavy. This separates node composition from topology. A graph population can overproduce Sheet nodes without also having the highest cycle counts.

Table 8: PROTEINS cycle counts (mean simple cycles per graph, exact). Both generators under-produce 3-cycles relative to real and over-produce 5- and 6-cycles, but the relative orderings differ by class.

Class	Source	3-cycles	4-cycles	5-cycles	6-cycles
0 (Enzyme)	real	24.79	34.58	41.02	65.97
	1-GNN gen	15.93	45.19	95.59	204.31
	1-2-GNN gen	9.07	25.21	86.52	272.20
1 (Non-Enzyme)	real	15.61	21.55	21.94	31.45
	1-GNN gen	11.08	33.56	78.70	189.72
	1-2-GNN gen	11.62	16.45	49.96	131.38

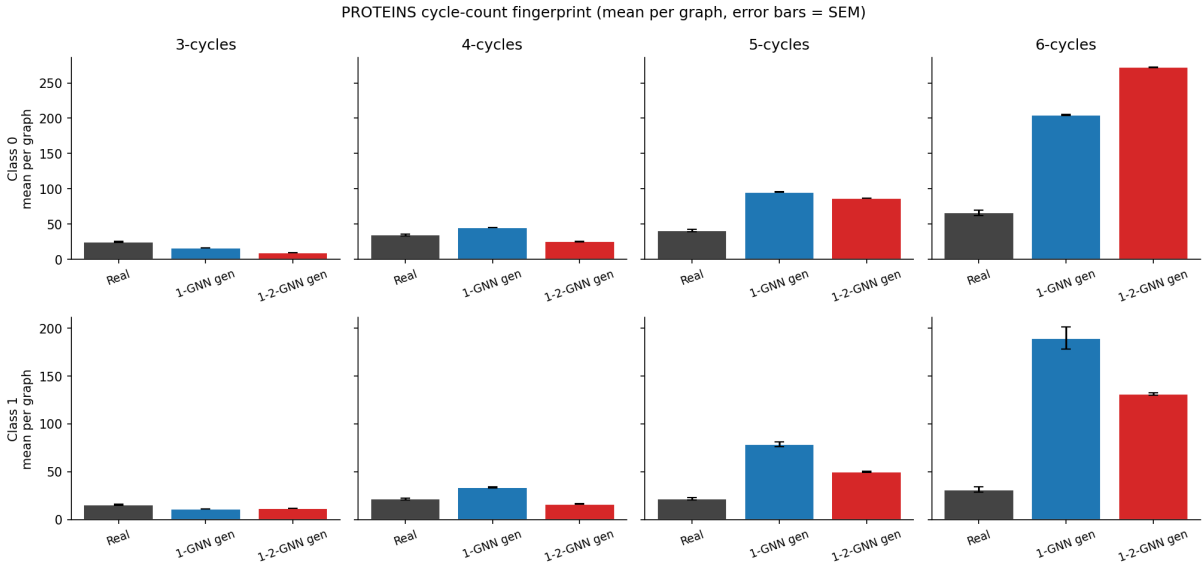


Figure 9: PROTEINS simple-cycle counts per graph, aggregated over 1000 generated graphs per cell. Error bars are the standard error of the mean. Both generators over-produce 5- and 6-cycles relative to real; the strongest differences appear in the 6-cycle panels, with the 1-2-GNN highest in class 0 and the 1-GNN highest in class 1.

5.3 Summary of the comparison

Across both datasets, the generated graphs should be interpreted as classifier-specific prototypes rather than faithful random samples from the real data. The clearest differences are class-specific: 1-2-GNN Mutagen graphs on MUTAG are mostly rejected by the 1-GNN, while 1-GNN Enzyme graphs on PROTEINS are mostly rejected by the 1-2-GNN.

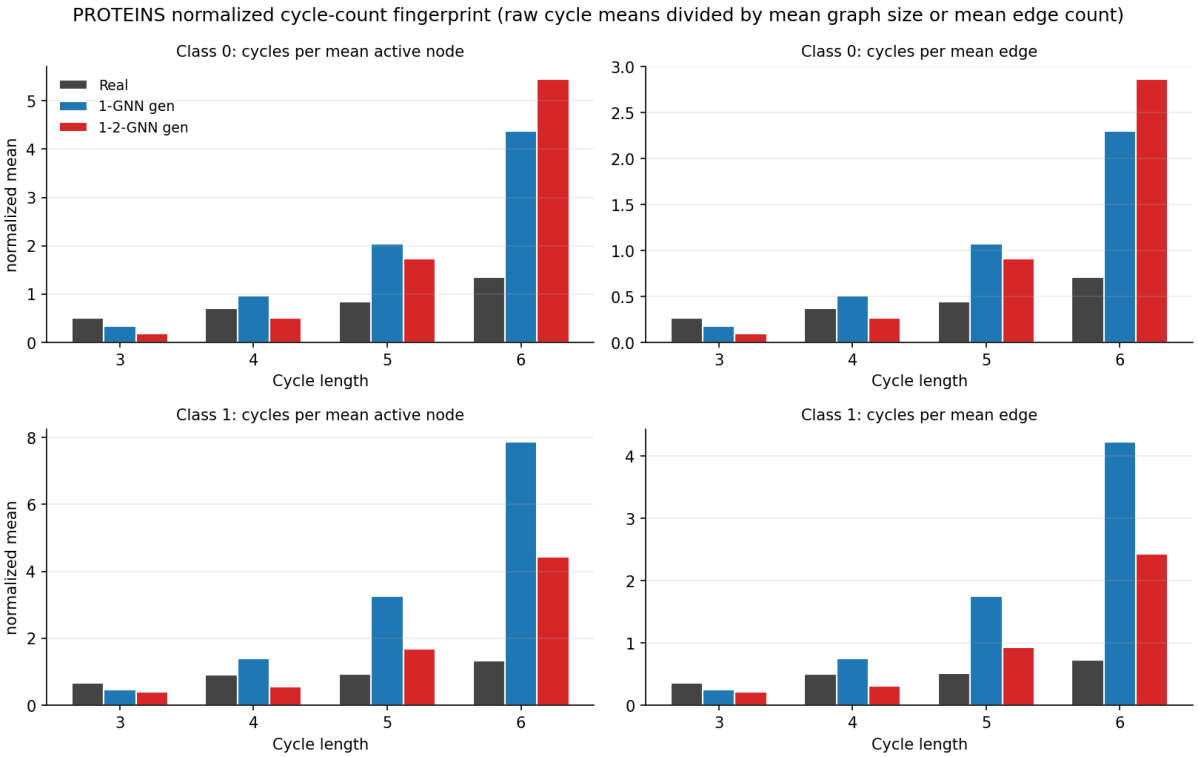


Figure 10: Normalized PROTEINS cycle-count fingerprint. The left panels divide each raw cycle mean by the mean active-node count; the right panels divide by the mean edge count. The normalization checks that the longer-cycle trend is not only a by-product of larger generated graphs.

The population fingerprints show why these failures are plausible: the two architectures shift graph size, node-type composition, edge-pair composition, and cycle structure in different directions. Higher-order message passing changes what the model treats as a convincing class example, but it does not make the generated explanations uniformly more realistic.

6 Discussion

6.1 Interpreting the generated prototypes

GIN-Graph isn't trained to draw random samples from the real class. It's trained to find graphs the classifier accepts as the target class. During training, the discriminator and three regularisers keep the search inside a constrained region: graph size, node type distribution, and average degree. At evaluation time, a separate degree-based validity gate filters graphs whose average degree falls too far from the real class statistics. So we should read the generated populations as the answer to "what convinces this classifier?", not as a substitute for the real data distribution.

The cross-model classification table is what we lean on most. In most cells the two classifiers agree: a graph one accepts, the other accepts too. There are two exceptions. On MUTAG class 1, the 1-2-GNN's generated Mutagen graphs are accepted by the 1-2-GNN itself but only by the 1-GNN 12.3% of the time. On PROTEINS class 0, it's the other way around: the 1-GNN's Enzyme graphs are accepted by the 1-GNN but only by the 1-2-GNN 14.0% of the time. So the disagreement isn't a blanket architectural split. It happens in one specific class per dataset.

This isn't surprising once you think about what the two architectures see. A 1-GNN never represents anything below the node level, so the structural distinctions it can encode are bounded by 1-WL: node labels plus the neighbourhood structure they sit in. A 1-2-GNN runs the same node-level pass and then builds an extra representation per node pair, with the edge indicator baked in. That gives the classifier a direct handle on edge-pair patterns and short cycles. The generator still only outputs nodes and edges, but the 1-2-GNN can pull on signal there that the 1-GNN can't read at all.

The MUTAG fingerprints show what this looks like in numbers. The 1-2-GNN's Non-Mutagen prototype is wildly over-halogenated compared to real Non-Mutagen molecules: +26 pp F, +11 pp Br, with the matching C-F and C-Br edges. Its Mutagen prototype does almost the opposite. Halogens drop close to zero, carbon goes up, and the edge mix tilts toward C-O and O-O. We weren't expecting the direction to flip between classes, but it tells us something useful: the divergence isn't "the 1-2-GNN likes more halogens" as a fixed bias. It's class by class.

The cycle counts are where the theory motivation actually shows up empirically. Real MUTAG molecules in our split have no triangles or 4-cycles. Both generators introduce some, so neither is a faithful molecular sample. But the 1-2-GNN produces fewer triangles than the 1-GNN in both classes, and a lot fewer in Mutagen (0.15 vs 1.01 per graph). That's the one result we can trace back to expressivity: 2-WL can tell triangle from non-triangle structure, 1-WL can't. We don't push the argument further than that. The 1-2-GNN over-produces 4-cycles in the same Mutagen prototype, so it's not as if the higher-order architecture makes everything more realistic. It changes which structural

features become discriminative.

Some populations overshoot. Sheet runs at 88.7% on PROTEINS Non-Enzyme (real: 56.2%), halogens at 41.7% on MUTAG Non-Mutagen (real: 4.1%). This isn't really a bug, more a consequence of the training objective. The generator only has to keep the classifier happy. As soon as it discovers a feature the classifier rewards, the GNN-guidance term keeps pushing on it. The structural regularisers are soft aggregate penalties, not hard distribution matching. They reduce drift in size, node-type mix, and average degree, but they do not force the generated population to match real class frequencies. So the features the classifier reads as discriminative drift past their real frequencies, and we get the overshoot.

PROTEINS is where degree-only validation runs out of resolution. Generated PROTEINS graphs hit the real mean degree, fine. They don't hit the real topology. In Figure 6 the real proteins are mostly chain-like, while the generated ones look like dense compact blobs. Some of that is built into the setup: we cap generation at $N = 50$, we don't enforce connectedness, and the validity gate only checks average degree. The generator can therefore arrange a realistic edge count in an unrealistic way, and the normalized cycle plots confirm the excess of long cycles isn't just an artefact of larger graphs.

PROTEINS Enzyme is where this really matters. The two generators land on similar Helix/Sheet proportions, and the 1-2-GNN's are actually closer to real than the 1-GNN's. Even so, the 1-2-GNN rejects 86% of the 1-GNN's Enzyme graphs. So the rejection can't just be about which nodes are present. The 1-2-GNN must be reading something else, probably the connectivity pattern between Helix and Sheet nodes that the 1-GNN doesn't have a way to control. Non-Enzyme reads differently. Both generators go Sheet-heavy, but the 1-2-GNN pushes further and tilts the edge mix toward S-S as well. The prototype shifts in composition and in topology.

Putting all of this together, the result is mixed. The 1-2-GNN does learn different class prototypes than the 1-GNN at the same accuracy, and on MUTAG class 1 the difference matches what 2-WL predicts: fewer triangles. The rest of the shifts are real but model-specific. We can describe them; we can't derive them from the WL hierarchy. The comparison wasn't designed to pick a winner. It's there to show that GIN-Graph surfaces differences accuracy hides, including differences that don't fit a clean expressivity story.

6.2 Limitations

Several limitations qualify the findings:

Single training run. We trained each model once, on a single split of the data (seed 42). MUTAG's test set has only 38 graphs, so misclassifying just one extra graph shifts accuracy by 2.6 percentage points, meaning the identical 89.5% accuracy on MUTAG could be a tie by coincidence rather than a real one. The same caveat applies to the small validity gap on MUTAG (82% vs. 78%, four graphs out of 100). This is why we focus on the population-level differences in the generated graphs rather than these headline numbers: the cross-classification rates and cycle counts in Section 5.2 are computed over 1000 generated graphs per cell, so they are stable against noise from the generation step. What we cannot rule out is variance from this single training run, so a different seed could shift the picture. PROTEINS has 224 test graphs and is more robust on the classifier side, but the class-specific pattern could still partly reflect properties of this particular

split.

Computational overhead. The step from 1-GNN to 1-2-GNN is not free. A 1-GNN processes n node embeddings per layer ($O(n \cdot \deg)$ on sparse graphs). A 2-GNN operates on the pair set $\binom{V}{2}$ of size $n(n-1)/2$, so each pair-level round scales as $O(n^2 \cdot \deg)$. On PROTEINS, where individual graphs reach 620 nodes, even the numba-accelerated CSR kernel of Section 3.2 costs $\approx 2.2\,\mathrm{s}$ per classifier forward pass, which is 326× the 1-GNN ($\approx 6.8\,\mathrm{ms}$). On MUTAG the ratio is 52×. Generation cost follows: producing one MUTAG graph takes $\approx 0.02\,\mathrm{s}$ with the 1-GNN and $\approx 1.48\,\mathrm{s}$ with the 1-2-GNN (74×), and on PROTEINS the gap is 36×. The dense wrapper of Section 3.5.4 is worse still, because it materialises all n^2 candidate pair features into a single tensor. This is unavoidable for autograd to route the gradient through the soft-attention weights, but it couples memory to the square of the generated graph size and is the reason the PROTEINS generation size is capped at $N = 50$. The $O(n^2)$ pair construction is intrinsic to the $k = 2$ step of the Weisfeiler–Leman hierarchy, so any extension to larger graphs has to either accept this cost, sample k-sets, or bypass the pair-level representation.

Generator–classifier mismatch. GIN-Graph was designed for standard 1-GNNs: its generator produces node features and an adjacency matrix, which is exactly what a 1-GNN reads. A 1-2-GNN instead makes decisions over node *pairs*, so the generator cannot target its representations directly. It can only influence them through its choices of nodes and edges. This indirect path still works (same-model agreement is $\geq 94\%$ for every configuration), but the lower prediction probability on PROTEINS Enzyme ($p = 0.564$ for the 1-2-GNN vs. 0.986 for the 1-GNN) is consistent with the mismatch.

Evaluation metrics. The validation score $v = (s \cdot p \cdot d)^{1/3}$ can be dominated by a single low component. The degree score, which measures structural realism via average degree alone, is a coarse proxy that does not capture finer structural properties like motif frequencies or degree distributions.

Reliance on a single generator. All comparisons between the 1-GNN and 1-2-GNN are mediated by one evaluation pipeline, the GIN-Graph generator. The differences we observe could therefore reflect what GIN-Graph can coax out of each classifier rather than properties intrinsic to the classifiers themselves. The influence flows from the generator to the comparison, not the other way around. Cross-comparing with another generator would help isolate which differences are real architectural divergences.

Connectivity and graph-size cap. Real MUTAG graphs are connected and real PROTEINS graphs are almost always connected, but the generated graphs are not constrained to remain connected after thresholding. Isolated nodes are removed before computing statistics, yet the remaining active graph can still split into several components. This matters most for cycle counts: separate dense components can increase the number of short and medium cycles without corresponding to one connected real graph. The PROTEINS cycle fingerprint also compares generated graphs capped at $N = 50$ against the real graphs that fit this cap, while the size table describes the broader real class distribution. The cycle results are therefore best read as evidence of structural drift in the generated populations, not as a perfect real-vs-generated motif audit.

6.3 Future Work

A natural extension is a higher-order–aware generator that produces pair-level (or more generally k-set–level) features directly, removing the output-space mismatch identified in Section 6.2 and giving a cleaner test of what a 1-2-GNN classifier encodes.

A second direction is extending the comparison to a 1-2-3-GNN, to test whether the architecture-specific divergence observed here continues up the k-WL hierarchy or is a feature of the 1-to-2 step alone.

A third direction, motivated by the overhead in Section 6.2, is to test whether a 1-GNN with a *positional encoding* (PE) can play the role of the 1-2-GNN at a fraction of the cost. PEs inject structural information into initial node features without pair-level message passing or the $O(n^2)$ blow-up; Grötschla et al. [Grötschla et al.(2026)] report that spectral encodings such as LapPE offer a favourable trade-off between WL distinguishability and preprocessing cost. Running the GIN-Graph pipeline on a 1-GNN+LapPE classifier would test whether the divergence we observe is driven by raw expressiveness or by the pair-level inductive bias specifically.

A fourth direction is to add explicit connectedness control and component-aware evaluation. The current generator is regularised by graph size, node-type distribution, and average degree, but not by connectedness. A future version should either penalise disconnected samples during generation or report all motif fingerprints on the largest connected component alongside the full active graph. This would make cycle-count comparisons cleaner, especially for PROTEINS where larger closures are part of the qualitative argument.

7 Conclusion

We asked whether the extra expressiveness of a higher-order k-GNN translates into a different class concept compared with a standard 1-GNN, and compared a 1-GNN against a hierarchical 1-2-GNN using GIN-Graph for model-level explanation generation across both classes of MUTAG and PROTEINS.

Two technical contributions made the comparison possible: a fully differentiable dense wrapper that keeps gradients flowing through both node features and the adjacency matrix of a hierarchical k-GNN, and a corrected validation score that replaces the original prediction-probability proxy with a real cosine similarity to a class centroid.

The central empirical finding is that the two architectures learn different ideas of what counts as a typical class member, even when their accuracy is the same. On MUTAG both classifiers reach 89.5%, but their generated populations diverge by class: the 1-2-GNN's Non-Mutagen graphs are halogen-heavy, while its Mutagen graphs suppress halogens and triangles. Cross-classification of 1000 generated graphs per cell confirms this: each model strongly rejects the other's graphs in one specific class per dataset (12.3% and 14.0% agreement). The MUTAG cycle counts give the one theory-aligned signal: the 1-2-GNN consistently produces fewer triangles than the 1-GNN, which is what pair-level message passing can distinguish but 1-WL cannot.

These differences do not amount to a blanket advantage: the generators preserve mean degree well but shift graph size, node-type composition, and cycle structure, and the

cross-classification asymmetries appear in only one class per dataset. The defensible reading is that higher-order message passing changes *what* kind of class prototype the classifier learns, not whether it is uniformly more realistic. This is exactly the kind of divergence that classification accuracy alone hides, and that model-level explanation generation makes visible.

References

[Borgwardt et al.(2005)] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, and H.-P. Kriegel. Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl_1):i47–i56, 2005.

[Debnath et al.(1991)] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. *J. Med. Chem.*, 34(2):786–797, 1991.

[Grötschla et al.(2026)] F. Grötschla, J. Xie, and R. Wattenhofer. Benchmarking positional encodings for GNNs and graph transformers. In *KDD*, 2026.

[Gulrajani et al.(2017)] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved training of Wasserstein GANs. In *NeurIPS*, 2017.

[Jang et al.(2017)] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with Gumbel-softmax. In *ICLR*, 2017.

[Morris et al.(2019)] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI*, 2019.

[Sun et al.(2025)] J. Sun, S. Pei, F. Zhang, and N. V. Chawla. GIN-Graph: A generative interpretation network for model-level explanation of graph neural networks. *Neurocomputing*, 2025.

Appendix A: Discriminator and WGAN-GP Details

This appendix expands the adversarial component summarised in Section 3.4: the discriminator architecture, the two-player training dynamics, and the derivation of WGAN-GP including why the gradient penalty rules out weight rescaling. The shorthand and notation are as in Section 3.4.

A.1 The Discriminator

The discriminator $\mathcal{D}$ is a graph neural network that maps a graph to a single scalar. It operates on dense batched inputs (node features $\mathbf{X} \in \mathbb{R}^{B\times N\times D}$ and adjacency $\mathbf{A} \in [0,1]^{B\times N\times N}$) so that the same forward pass handles both real and generated graphs. The architecture has three stages:

1. **Two dense GCN layers** with hidden dimension 128 and ReLU activations. Each layer computes $\mathbf{H}^{(t)} = \sigma(\hat{\mathbf{A}}\,\mathbf{H}^{(t-1)}\,\mathbf{W}^{(t)})$, where $\hat{\mathbf{A}}$ is the adjacency with self-loops added and symmetric degree normalisation applied.
2. **Mean pooling** across the node dimension: $\mathbf{h}_G = \frac{1}{N}\sum_{v=1}^{N} \mathbf{h}_v^{(2)} \in \mathbb{R}^{128}$. This produces a single graph-level vector per graph.
3. **Two-layer MLP** with LeakyReLU activation (negative slope $\alpha = 0.2$, as in Section 3.3): $\mathcal{D}(G) = \mathbf{w}^\top \mathrm{LeakyReLU}_{0.2}(\mathbf{W}\,\mathbf{h}_G) \in \mathbb{R}$. In the discriminator the LeakyReLU choice is standard for GAN training, where ReLU's zero-gradient half-plane can cause dead units that stop contributing signal to the generator during adversarial updates.

Higher values of $\mathcal{D}(G)$ indicate that the graph appears real; lower values indicate that it appears generated. There is no sigmoid or softmax at the output, so $\mathcal{D}(G)$ is not a probability. In a standard binary classifier, the scalar would pass through a sigmoid $\sigma(x) = 1/(1+e^{-x})$ to produce a value in $[0, 1]$. Here the output is left unbounded. The reason is explained below in the comparison with vanilla GANs.

The discriminator is distinct from the pretrained k-GNN classifier Φ used in $\mathcal{L}_{\text{GNN}}$: $\mathcal{D}$ is trained from scratch during GIN-Graph training and scores graphs on realism, while Φ is frozen and scores graphs on class membership.

A.2 The Two-Player Setup

The generator $\mathcal{G}$ and the discriminator $\mathcal{D}$ are trained in alternating steps:

- The **discriminator** receives a batch of real graphs from the training set and a batch of generated graphs from the current generator, and updates its weights to increase scores on real graphs and decrease scores on generated graphs.
- The **generator** produces a batch of graphs, passes them through the discriminator, and updates its weights to increase the scores the discriminator assigns.

Neither network is trained to convergence; they co-evolve. In each training step, the discriminator updates once, then the generator updates once ($n_{\text{critic}} = 1$). This alternation is necessary. If $\mathcal{D}$ were trained to convergence against a fixed generator, its output would be correct (low on all generated graphs, high on all real graphs) but uninformative as a gradient signal: the generated region of input space would become flat, so small changes to a generated graph would not change $\mathcal{D}$'s score, and the generator would receive no direction for improvement. Keeping $\mathcal{D}$ partially trained ensures that its scores still vary across the region where the generator's current outputs lie.

A.3 Discriminator and Generator Losses

In each training step, we draw a batch of $B = 64$ real graphs G from the training set and a batch of B fake graphs $\tilde{G}$ from the generator. The discriminator minimises

$$\mathcal{L}_D = \underbrace{\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]}_{\text{score on fake}} - \underbrace{\mathbb{E}_{G\sim p_{\text{data}}}[\mathcal{D}(G)]}_{\text{score on real}} + \underbrace{\lambda_{\text{GP}}\,\mathbb{E}_{\hat{G}}\Big[(\|\nabla_{\hat{G}}\mathcal{D}(\hat{G})\|_2 - 1)^2\Big]}_{\text{gradient penalty}}, \tag{32}$$

with $\lambda_{\text{GP}} = 10$, where each expectation $\mathbb{E}[\cdot]$ is in practice the average over one batch. Consider the first two terms:

- $\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]$ is the average score the discriminator assigns to generated graphs. The discriminator minimises $\mathcal{L}_D$, so it pushes this term down, assigning *low* scores to generated graphs.
- $-\mathbb{E}_{G\sim p_{\text{data}}}[\mathcal{D}(G)]$ is the negative of the average score on real graphs. Minimising this term pushes the average score on real graphs *up*.

Together, these two terms encode one instruction: push real scores up, push generated scores down. The third term, the gradient penalty, is a stability device explained below.

The generator's adversarial loss in Equation (19) is the negation of the discriminator's "score on fake" term. The generator wants to maximise the score the discriminator assigns to its outputs. Since optimisers minimise by convention, we negate: minimising $-\mathbb{E}[\mathcal{D}(\tilde{G})]$ is equivalent to maximising $\mathbb{E}[\mathcal{D}(\tilde{G})]$. Real graphs do not appear in $\mathcal{L}_{\text{GAN}}$. The generator never sees real data directly. It learns about the real distribution only through the gradient signal from $\mathcal{D}$: real graphs train $\mathcal{D}$, and $\mathcal{D}$'s gradient informs the generator how to adjust its outputs. This indirect chain, from real data to $\mathcal{D}$'s parameters to $\mathcal{D}$'s gradient to the generator's parameters, is the core mechanism of GANs.

A.4 Why WGAN-GP and Not a Vanilla GAN

In a vanilla GAN, the discriminator ends in a sigmoid, so its output is a value in $[0, 1]$ interpreted as the probability of the input being real. The problem is gradient saturation. Once $\mathcal{D}$ can reliably separate real from generated graphs, the sigmoid output for generated graphs is pushed toward 0. The derivative of the sigmoid $\sigma(x) = 1/(1 + e^{-x})$ is $\sigma(x)(1 - \sigma(x))$, which approaches zero when $\sigma(x) \approx 0$ or $\sigma(x) \approx 1$. The generator's weight updates are computed via the chain rule as a product of derivatives, one of which is this sigmoid derivative. A near-zero factor makes the entire product near-zero, and the generator stops learning. This is the core difficulty of vanilla GAN training.

WGAN removes the sigmoid. The discriminator's output is an unbounded real number with no flat region. When $\mathcal{D}$ becomes strong, its output for generated graphs continues decreasing, and the derivative remains non-degenerate, so the generator continues to receive a gradient signal.

Removing the sigmoid introduces a new problem: without any bound on $\mathcal{D}$'s output, it can reduce its loss without limit by scaling its weights up. Doubling all weights roughly doubles the output gap between real and generated scores. The gradient penalty in Equation (32) prevents this. The term

$$\lambda_{\text{GP}}\,\mathbb{E}_{\hat{G}}\Big[(\|\nabla_{\hat{G}}\mathcal{D}(\hat{G})\|_2 - 1)^2\Big]$$

penalises deviation of the gradient magnitude of $\mathcal{D}$'s output (with respect to its input $\hat{G}$) from 1. Here $\hat{G} = \epsilon G + (1 - \epsilon)\tilde{G}$ with $\epsilon \sim \text{Uniform}(0, 1)$ is a random interpolation between a real and a generated graph, so the penalty is evaluated at points between the two distributions. The coefficient $\lambda_{\text{GP}} = 10$.

This constraint has two effects. First, $\mathcal{D}$ cannot reduce its loss by weight rescaling. Consider a linear discriminator $D_\psi(G) = \psi^\top \text{vec}(G)$ as a tractable case. Then $\nabla_G D_\psi = \psi$, so $\|\nabla_G D_\psi\|_2 = \|\psi\|_2$. Rescaling the weights $\psi \mapsto k\psi$ rescales the gradient norm to $k\|\psi\|_2$, and the gradient-penalty term becomes $(k\|\psi\|_2 - 1)^2$, which grows quadratically in k and dominates $\mathcal{L}_D$ for large k. For the deep LeakyReLU discriminator used here,

the activations are positively homogeneous ($\text{LeakyReLU}_\alpha(c\,x) = c\,\text{LeakyReLU}_\alpha(x)$ for $c \geq 0$), so scaling every weighted layer by k scales the output by k^L with L the number of weighted layers, and the gradient norm scales the same way. The penalty therefore forbids rescaling as a route to lower $\mathcal{L}_D$: any rescaling inflates the penalty much faster than it can inflate the adversarial gap between real and generated scores. Second, the gradient flowing from $\mathcal{D}$ into the generator remains at a bounded magnitude, neither vanishing (the vanilla GAN failure) nor exploding (the unconstrained WGAN failure).

In summary: a vanilla GAN's sigmoid saturates when $\mathcal{D}$ is confident, killing the generator's gradient. WGAN-GP removes the sigmoid and adds a gradient penalty that keeps the gradient magnitude near 1. The generator therefore receives a non-degenerate learning signal regardless of $\mathcal{D}$'s strength.

Appendix B: Structural Regularisation Derivations

This appendix derives the degree, size, and atom-type penalties whose combined loss is stated in Equation (22). The regulariser decomposes as $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{degree}} + \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{type}}$, where $\mathcal{L}_{\text{degree}}$ penalises deviation from the class mean of average degree, $\mathcal{L}_{\text{size}}$ penalises deviation from the class mean graph size, and $\mathcal{L}_{\text{type}}$ penalises deviation from the class atom-type distribution. These targets are precomputed once from the real training set and remain fixed during training.

B.1 Size penalty

The generator outputs a fixed $N \times N$ matrix with no flag marking which slots are active. A slot is effectively active if it has at least one edge to another slot, so the natural notion of graph size is the number of slots with at least one edge.

Define the soft degree of slot v as the row-sum of the soft adjacency matrix:

$$d_v = \sum_{u=1}^{N} \tilde{A}_{vu}.$$

A slot with no edges has $d_v \approx 0$; a slot with one edge has $d_v \approx 1$; a slot with three edges has $d_v \approx 3$. The naive active-slot count would use an indicator:

$$\hat{n}_{\text{naive}} = \sum_{v=1}^{N} \mathbf{1}[d_v > 0.5],$$

treating a slot as active if it has at least half an edge of connection. This does not work for training: the indicator is a step function with derivative zero almost everywhere and undefined at the step. Any gradient flowing from $\mathcal{L}_{\text{size}}$ through this path collapses to zero.

We therefore replace the step with a smooth approximation. The logistic sigmoid $\sigma(x) = 1/(1+e^{-x})$ approximates a step at $x = 0$. To place the transition at $d_v = 0.5$ and sharpen it, we shift the argument by 0.5 and scale by 10:

$$\hat{n} = \sum_{v=1}^{N} \sigma\big(10 \cdot (d_v - 0.5)\big).$$

The multiplier 10 is steep enough that slots with $d_v < 0.3$ contribute near zero and slots with $d_v > 0.7$ contribute near one:

d_v	0	0.3	0.5	0.7	1.0
$\sigma(10(d_v - 0.5))$	0.007	0.12	0.50	0.88	0.99

The expression approximates the active-slot count on the forward pass while providing non-zero gradients everywhere on the backward pass.

With $\hat{n}$ (soft count of active slots) and $\bar{n}_c$ (mean node count of real class-c graphs, pre-computed once), the size penalty is

$$\mathcal{L}_{\text{size}} = \lambda_{\text{size}} \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2 .$$

Two design choices matter. The squared error is smooth at zero with derivative vanishing as the error approaches zero, so the generator settles into the target without oscillation; the absolute value has a non-differentiable corner at zero and a constant-magnitude gradient. The division by $\bar{n}_c$ converts the error into a relative error, making the penalty scale-invariant across datasets: a 5-node error on MUTAG class 0 ($\bar{n}_c \approx 13.9$) is a $\sim$36% failure, while the same error on a dataset with $\bar{n}_c \approx 40$ is only $\sim$12.5%. Without this normalisation, λ_{size} would need to be retuned for every new dataset.

The coefficient $\lambda_{\text{size}} = 10$ is ten times larger than λ_{type} because a size violation breaks the graph structurally (a 50-node molecule where the class mean is 14 is unrecognisable), while atom-type violations are a softer failure.

B.2 Degree penalty

The degree penalty uses the same soft adjacency matrix. Let

$$\hat{m} = \frac{1}{2} \sum_{u=1}^{N} \sum_{v=1}^{N} \tilde{A}_{uv}$$

be the soft edge count of the generated graph. The active-node count used for this term follows the implementation and treats a node as active once its soft degree exceeds 0.5:

$$\hat{n}_{\text{hard}} = \max\left(\sum_{v=1}^{N} \mathbf{1}[d_v > 0.5], 1 \right).$$

The generated average degree is then

$$\hat{d} = \frac{2\hat{m}}{\hat{n}_{\text{hard}}}.$$

With μ_c^d and σ_c^d denoting the mean and standard deviation of real average degrees in class c, the penalty is

$$\mathcal{L}_{\text{degree}} = \lambda_{\text{degree}} \left(\frac{\hat{d} - \mu_c^d}{\sigma_c^d} \right)^2 ,$$

with $\lambda_{\text{degree}} = 1$. Unlike the size penalty, this hard count is not the quantity being matched directly. It only rescales the differentiable edge count in the numerator. The active-node denominator keeps the statistic comparable to the real average degree, which is computed over non-empty graphs. This term is part of the generator training loss. At evaluation time, degree also appears separately in the validation score and the hard validity gate of Section 3.6.

B.3 Atom-type penalty

For each class c, the target distribution $p^c = (p_1^c, \ldots, p_D^c)$ is the frequency of each atom type in real class-c graphs, where p_i^c is the fraction of atoms of type i. This is a probability vector: entries in $[0, 1]$ summing to 1. For MUTAG class 0, $p^c \approx (0.85, 0.08, 0.04, \ldots)$ across the $D = 7$ atom types, with carbon dominating. Computed once, stored, reused.

The generated frequencies are computed from $\tilde{X}$. Each row of $\tilde{X}$ is nearly one-hot after Gumbel-Softmax, so summing column i counts how many slots selected atom type i:

$$\hat{p}_i = \frac{1}{N} \sum_{v=1}^{N} \tilde{X}_{v,i}.$$

Both $\hat{p}$ and p^c are probability distributions over the same D atom types. The penalty is the squared L2 distance between them, summed across types:

$$\mathcal{L}_{\text{type}} = \lambda_{\text{type}} \sum_{i=1}^{D} (\hat{p}_i - p_i^c)^2,$$

with $\lambda_{\text{type}} = 1$. The coefficient is smaller than λ_{size} because the sum already accumulates D squared errors and reaches meaningful magnitudes with a unit weight.

The natural distance between probability distributions is KL divergence, not L2. We use L2 for three reasons:

1. KL is undefined when $p_i^c = 0$, because the formula contains $\log(\hat{p}_i / p_i^c)$. On MUTAG, some classes contain zero instances of some atom types, so this is a concrete problem.
2. KL is asymmetric: $D_{\text{KL}}(\hat{p} \,\|\, p^c) \neq D_{\text{KL}}(p^c \,\|\, \hat{p})$. The two directions favour different failure modes and selecting one requires arbitrary justification. L2 is symmetric by construction.
3. KL has a gradient with a $1/p_i^c$ factor that explodes for rare atom types. The L2 gradient is $2(\hat{p}_i - p_i^c)$, bounded and stable.

B.4 Gradient path

$\mathcal{L}_{\text{reg}}$ has the shortest gradient path of the three loss components. Its gradients flow from scalar arithmetic on $(\tilde{A}, \tilde{X})$ through the Gumbel-Softmax backward pass into the generator MLP, without invoking the discriminator or the classifier. It is therefore the cheapest term per training step, since every other term requires a full network forward and backward pass while $\mathcal{L}_{\text{reg}}$ is closed-form. The target class enters only through the precomputed statistics μ_c^d, σ_c^d, $\bar{n}_c$, and p^c, so changing the target class requires no change to the computation graph.